

SYSTEM VERILOG

6/02/25

- Verilog was developed first
- Verilog basing on C they did some modifications and developed the verilog.
- Concept of Time: Basing delay and clk we will support this concept of time
- concept of state: In hardware descriptive language we have concept of states
- concept of concurrent execution: we have always and initial Block to about concept of concurrency.
- concept of structure: Through port connection

Drawbacks in verilog:

- we have less number of constructs
- we don't have ops concepts.
- Modularity is not there in verilog
- we need to develop entire test bench in single file (b/w module and end module)
- code reusability

for rectifying the drawbacks they develop a language called system verilog

system verilog:

- It is an advanced version of verilog
- In system verilog = verilog + advanced features
- In verilog we have only 4 state variables $\rightarrow 0, 1, x$ and z
- In system verilog we have 4 state variables and 2 state variables
4 state means $0, 1, x, z$
2 state means $0, 1$

Data types in system Verilog

- Net
 1. reg
- wire
 2. Integer
- reg
 3. real
 4. Time
 5. Real time

• In verilog we don't have string datatype

- declare a string with 10 characters
- By using reg we need to declare strings

```
reg [10*8-1:0] str;
```

Pass the string through user define

```
$value$plusargs ("str = %0s", str);
```

System Verilog data types:

- | | | |
|------------------|-----------------|---------------|
| 1. wire | 7. Integer | 13. Real time |
| 2. reg | 8. Long integer | 14. Real |
| 3. Bit | 9. Short int | 15. String |
| 4. Byte | 10. Long int | 16. Void |
| 5. Logic | 11. Real | |
| 6. Short integer | 12. Time | |

1. Wire

- It continuously drives the data from inputs to outputs
- Default value will be z
- It can drive one bit of data
- wire will be 4 state variable

3. Bit

- It stores 1 bit of data
- It is a 2 state variable
- Default value of bit 0
- bit will be unsigned

2. Reg

- It is descriptly drives the data
- It stores 1 bit of data
- Default value will be x
- reg will be 4 state variable
- reg will be unsigned

4. Byte

- It stores 8 bits of data
- It is a 2 state variable
- Default value will be 0
- It is signed

5. Logic

- It is improved version of reg
- It is used to declare the variables in continuously drive data and gate modules
- It will be a 4 state variable
- default value will be x

7. short integer: same as integer but it stores 16 bits of data

8. Long integer: same as integer but it stores 64 bits of data

9. ~~short int~~ short int: same as int but it stores 16 bits of data

10. Long int: It is as same as int but it will store 64 bits of data

11. Int: It stores 32 bits of data

It is signed

2 state variable

default value will be 0

12. Real: It

• It stores 64 bits of data

• default value will be 0

• It is 2 state variable

• It is signed

14. Real time

• It will tell the simulation time

• It stores 64 bits of data

• default value will be 0

• It is also 2 state variable

15. void: If there is ~~a collection of~~ no return type means we need to use this data type

16. String: String is a collection of character enclosed with "

Syntax: string string-name;

6. Integer

• It stores 32 bits of data

• It is signed data type

• default value will be x

• It is a 4 state variable

13. Time

• It will tell the simulation

• It stores 64 bits of data

• It is a 4 state variable

• default value will be x.

Methods in strings:

1. Length: It tells about the size of the string

Syntax: `string-name.length();`

2. `putc`: It is used to insert the character into the string basing on the index

Syntax: `string-name.putc(i, 'c');` // i means index and c means character

3. `getc`: It is used to get a character from the string basing on the index

Syntax: `string-name.getc(i);` // i means index

4. `toupper`: It is used to convert all the characters into uppercase

Syntax: `string-name.toupper();`

5. `tolower`: It will convert all the characters into lowercase

Syntax: `string-name.lower();`

6. `compare`: It will compare two strings

Syntax: `String-name1.compare(String-name2);`

- It is case sensitive

- It will treat uppercase as uppercase

- Lower case as lower case

7. `icompare`: It will also used for comparing the strings

Syntax: `String-name1.icompare(String-name2);`

- it will be case insensitive

- it will treat Upcase and Lower case characters are same

8. Concatenation: It is used to concatenate 2 or more strings

- by using this `{ }` we can perform concatenation.

Syntax: `string-name3 = { string-name1, string-name2 };`

9. `substring`: It is used to create substring from the string

Syntax: `String-name.substring(i, j);`

// i will be starting index and j will be ending index

Ex: 5670
 $S_1 = VLSI$
 $S_2 = LUPU$
 651864

*Module 1b;

string str1, str2, str3, str4, str5;

int n;

byte c;

bit a, b;

initial begin

str1 = "VLSI-GURU";

n = str1.length();

\$display("length of the string = %0d", n); : length of the string = 9

str1.putc(6, "K");

\$display("String = %0s", str1); : String = VLSI-gKRU

c = str1.getc(4);

\$display("character = %0s", c); : character = -

\$display("string = %0s", str1);

str2 = str1.toupper();

→ : string = VLSI-GKRU

\$display("String = %0s", str2);

: string = VLSI-GURU

str3 = str2.toLowerCase();

\$display("string = %0s", str3); : string = VLSI-gKRU

a = str1.compare(str3);

\$display("a = %0b", a); : a = 0

b = str1.compare(str2);

: b = 1

\$display("b = %0b", b);

if(! (str1.compare(str3))) \$display("strings str1 and str3 are same");

else \$display("strings str1 and str3 are not same"); are same

if(! (str1.compare(str2))) \$display("strings str1 and str2 are same");

else \$display("strings str1 and str2 are not same"); Not same

if(! (str1.compare(str3))) \$display("strings str1 and str3 are same");

else \$display("strings str1 and str3 are not same"); are same

if(! (str1.compare(str2))) \$display("strings str1 and str2 are same");

else \$display("strings str1 and str2 are not same"); are same

str4 = {str1, str2};

str5 = str1.substr(2, 6);

\$display("str4 = %0s", str4);

\$display("str5 = %0s", str5);

str4 = VLSI-gKRVLSI-GURU

end module str5 = SI-gK

Q. Declare a string with user defined data and find the length of string put a character into the string in any index and that get a character from the string and convert that string into uppercase and store that string in another string and compare both the string by using compare and icompare.

* If i want to generate random strings means \$random won't work
\$random will generate 32 bit value and we are using string

// conflict is data type
→ i need to use type casting
datatype ' (value);

// for generating random strings we have

→ \$formatf

→ %sformat

* Module tb;

string str;

int a;

initial begin

repeat (10) begin

a = \$urandom-range(65,100);

* str = string'(a); → int → string conversion

\$display("str = '%s'", str);

type casting static

end

end

endmodule

// Error (suppressible): (vsim-7070) Assigning a packed type 'reg signed[31:0]' to a string requires a cast.

// # str = "A" vsf

// # str = A

o/p: str = A

str = k

str = k

str = L

str = W

str = k

str = W

str = L

str = L

str = k

* module tb;

wire a;

reg b;

logic c;

byte e;

integer f;

int g;

short int h;

long int i;

real j;

short real k;

time l;

realtime m;

initial begin

\$display 1 "default value of wire = 1'b", a); 2

" " " " reg = 1'b", b); x

logic = 1'b", c); x

bit = 1'b", d); 0

byte = 1'b", e); 00000000 -8

integer = 1'b", f); xxx --- -32

int = 1'b", g); 000 --- -32

short int = 1'b", h); 000 --- -16

long int = 1'b", i); 0000 --- -64

real = 1'b", j); 0000 --- .64

short real = 1'b", k); 0000 --- 32

time = 1'b", l); xxx --- -64

realtime = 1'b", m); 0000 --- .32

end

endmodule

e/p: string;

module tb;

string str;

initial begin

str = "vlsi_guru";

\$display 1 "str = 1'os", str); end endmodule

// # str = vlsi_guru

Verilog

1. Verilog is hardware descriptive language
2. Less constructs
3. Don't have modularity concept
4. Don't have ops concepts
5. Less reusability and readability
6. Only 4 state variables
states: 0, 1, x and z

System Verilog

1. System Verilog is hardware verification
2. More number of constructs
3. Have modularity concept
4. Have ops concept
5. Have more reusability and more readability
6. 4 state and 2 state variables
4 state - 0, 1, x, and z
2 state - 0 and 1

Random String Generations:

We have 2 ways:

1. By using static casting or type casting we will generate random strings
2. By using system task and functions

1. \$sformatf → function
2. \$fformat → task

// Generate 10 random strings.

Module tb

String str;

int value;

initial begin

for (int i=0; i<10; i++) begin

value = \$urandom_range(65, 97);

// \$sformat (str, "f.os", value);

str = \$sformatf("f.os", value);

\$display("str = f.os", str);

end

end

endmodule

o/p: str = S

str = K

str = T

str = O

str = Q

str = I

str = Z

str = K

str = R

str = Z

str = Z

end

Vectors

1. It will represent more than one bit of data
2. It will store multiple bits of data in single locations
3. We can perform all the operations in vectors directly
4. We can access vectors directly
5. Vectors we can declare only reg or wire

Arrays

1. Stores multiple bits of data in multiple locations
2. Very few operations are possible directly
3. Perform all the operations means we need to consider index wise
4. Access arrays through index wise.
5. Syntax: `<data-type> <variable-name> [<size>];`
6. We can declare all the data type

Q. In the same question assign vectors 5th position as 1 and 7th position as 1 and assign random to the array b/w 10 to 20

```
reg [9:0] a;
integer array [9:0];

initial begin
  a = 0;
  a = 10'b0;
  a[6] = 1'b1;
  a[8] = 1'b1;
  a = 10'b00101 - 00000;
  for (int i = 0; i < 10; i++) begin
    array[i] = $URandom_range(10, 20);
  end
end
```

* System Verilog arrays

• we have only arrays there is no vector name in system verilog

11 How the array can be classified → 2 ways

1. ^{when} ~~but~~ memory is allocated
2. where memory is allocated

11 when memory is allocated

- In this type basing on the compilation and run time divide the arrays
 - If i allocated memory during compilation means it is known as static array
 - Allocating memory during run time it is known as dynamic
1. static arrays
 2. dynamic arrays

11 where memory is allocated

- where we are allocating memory before the variable name or else after the variable name
- Before the variable name means packed array
- After the variable name means unpacked arrays

Packed arrays

1. same as vectors in verilog
2. stores multiple bits of data in single location
3. Access directly
4. Reg or wire ~~are~~ used in
5. packed arrays perform all the operations.

Syntax: <data-type> <size>
 <variable-name>

unpacked arrays

1. same as arrays in verilog
2. stores multiple bits of data in ^{multiple} single address location
3. can't access directly. Access through Index
4. use all the data types
5. can't all the operations directly. To perform all the operations Then consider Index.

Q: Write a code for unique even values b/w 10 to 40 of the array whose size is 10

```

Module tb;
int arr[9:0];
int value;
reg exist-f;
initial begin
  for(int i=0; i<10; i++) begin
    value = $urandom_range(10,40);
    exist-f = 0;
    if(value % 2 == 0) begin
      for(int j=0; j<10; j++) begin
        if(arr[j] == value) begin
          j = i;
          exist-f = 1;
        end
      end
    end
    if(exist-f == 0) begin
      arr[i] = value;
      i++;
    end
  end
end
$display ("arr = %p", arr);
end
endmodule

```

o/p: arr = {36, 34, 24, 25, 37, 15, 23, 38, 33, 13}

arr = {40, 30, 26, 18, 32, 28, 36, 34, 24, 38}

* When memory is allocated

Static arrays:

- Allocate memory during compile time
- Size of the array will be fixed

Syntax: <data-type> <variable-name> [<size>]

Ex: Declare a static integer array of size 20 with name as arr

```
integer arr[19:0];
```

```
integer arr[0:19];
```

integer arr[20]; // these will be possible in system verilog

=) initial begin

```
for(int i=0; i<20; i++) begin
```

```
  arr[i] = $urandom_range(10,20);
```

```
  arr1[i] = $urandom_range(10,20);
```

```

    arr2[i] = $urandom_range(10,20);
  end $display("arr = %0d", arr);
    "
    arr1;
  end

```

Dynamic arrays: Allocate memory during run time
Dynamic array adjust size

How we will allocate memory

For allocating a memory we will consider one construct "new".

Syntax: <datatype> <variable name> [];

Ex: integer dynamic arrays

```
integer arr[ ];
```

```
// reg dynamic array
```

```
reg a[ ];
```

```
// 5 bit dynamic arrays
```

```
reg [4:0] a[ ];
```

// How to allocate memory

Allocate dynamic arrays size as 10

```
int a[ ];
```

```
initial begin
```

```
a = new[10];
```

```
end
```

Q: write a code for unique even values b/w 10 to 40 of the array whose size is 10.

```
module tb;
```

```
int arr[9:0];
```

```
int value;
```

```
reg exist-f;
```

```
initial begin
```

```
for(int i=0; i<10; )begin
```

```
value = $urandom-range(10,40);
```

```
exist-f = 0;
```

```
if(value % 2 == 0)begin
```

```
for(int j=0; j<i; j++)begin
```

```
if (arr[j] == value)begin
```

```
j = i;
```

```
exist-f = 1;
```

```
end
```

```
end
```

```
if(exist-f == 0)begin
```

```
arr[i] = value;
```

```
i++;
```

```
end
```

```
end
```

```
end
```

```
$display("arr=%p",arr);
```

```
end
```

```
endmodule
```

Dynamic array; static array

```

Module tb;
integer arr[19:0]; // 19, 18, 17, 16
integer arr1[0:19]; // 0, 1, 2, 3, 4
integer arr2[20]; // 0, 1, 2, 3...
initial begin
    for (int i=0; i<20; i++) begin
        arr[i] = $urandom-range(10, 20);
        arr1[i] = $urandom-range(10, 20);
        arr2[i] = $urandom-range(10, 20);
    end
    $display("arr = %p", arr);
    $display("arr1 = %p", arr1);
    $display("arr2 = %p", arr2);
end
endmodule

```

o/p: arr = { 12, 18, 13, 14, 19, 13, 14, 12, 19, 20, 14, 19, 20, 15, 20, 14, 13, 13, 17 }
 arr1 = { 20, 15, 20, 19, 12, 15, 13, 13, 13, 10, 15, 10, 16, 15, 18, 14, 14, 10, 18, 10 }
 arr2 = { 18, 19, 16, 13, 20, 12, 11, 13, 19, 11, 20, 14, 11, 15, 10, 11, 12, 15, 11, 12 }

Dynamic array methods:

NEW, SIZE, DELETE

NEW: It is used to allocate size of an array

Syntax: array-name = new[size]; Ex: da = new[10];

SIZE: It tells about the size of the arrays

Syntax: n = array-name.size();

DELETE: It will delete complete array elements

Syntax: n = array-name.delete();

array1.

```

Module tb;
int arr[];
initial begin
    ① arr = new[10];
    for (int i=0; i<10; i++) begin
        arr[i] = $random % 10;
    end

```

dynamic array

```

Module tb;
int a[];
initial begin
    a = new[10];
    // a = new[20];
    if (a.size()) begin
        $display("array having size");
        $display("array size = %d", a.size());
    end
    else $display("size is not allocated");
end
endmodule

```

o/p: 1 size is not allocated
 // array having size
 array size = 10
 array having size
 array size = 20

08/02/25

```

    $display("arr = %p", arr);
    $display("array-size = %d", arr.size()); -②
    arr.delete(); -③
    $display("arr = %p", arr);
    $display("array-size = %d", arr.size());

```

end
endmodule

O/P: arr = {8, -9, -9, -9, 7, 7, -1, -4, 1, 9}

array-size = 10

arr = {}

array-size = 0

Q: Declare 2 arrays and assign random values in the range 100 to 200 to the first array and copy those values into 2nd array
both array size will be 10

module tb;

int arr1[];

int arr2[];

// real arr2[];

initial begin

arr1 = new[20];

arr2 = new[10];

for (int i = 0; i < 20; i++) begin

arr1[i] = \$urandom-range(100, 200);

// arr2[i] = arr1[i]; → No need of same size → index wise

end

arr2 = arr1; → same size without index → Ex: a = new[5]
b = new[5]

\$display ("arr1 = %p", arr1);

\$display ("arr2 = %p", arr2);

end

endmodule

O/P: arr1 = {102, 188, 102, 167, 145, 182, 117,
127, 168, 130}

arr2 = {102, 188, 102, 167, 145, 182, 117, 127,
168, 130}

→ arr1 = {102, 188, 102, 167, 145, 182, 117, 127, 168, 130}

arr2 = {102, 188, 102, 167, 145, 182, 117, 127, 168, 130}

* copying one array elements to another array means

* -> Need to maintain the same data type and same size of the array.

module tb;

int arr[];

initial begin

arr = new[10];

\$display ("array-size = %d", arr.size());

\$display ("... increasing the size ...");

arr = new[20];

\$display ("array-size = %d", arr.size());

\$display ("... decreasing the size ...");

arr = new[5];

\$display ("array-size = %d", arr.size());

end

endmodule

O/P: array-size = 10

increasing the size

array-size = 20

decreasing the size

array-size = 5

Q: Consider a dynamic arrays int and size of both arrays same to assign random values between the range 10,50 for both you need to increase the size of the second array and assign my first array elements in increased size.

```

Module tb;
int arr1[];
int arr2[];
int arr3[];
initial begin
    arr1 = new[10];
    arr2 = new[10];
    arr3 = new[10];
    for (int i=0; i<10; i++) begin
        arr1[i] = $urandom-range(10,50);
        arr2[i] = $urandom-range(10,50);
    end
    $display("arr1 = %p", arr1);
    $display("arr2 = %p", arr2);
    arr3 = arr2;
    arr2 = new[20];
    for (int i=0; i<20; i++) begin
        if (i<10) arr2[i] = arr3[i];
        else arr2[i] = arr1[i-10];
    end
    $display("arr3 = %p", arr3);
    $display("arr2 = %p", arr2);
end
endmodule

```

arr1 = {29, 34, 32, 19, 43, 12, 18, 23, 29, 12}
 arr2 = {18, 26, 13, 30, 46, 25, 28, 30, 17, 20}
 → arr1 = Same
 arr2 = {0, 0, ..., 0} - 20 '0's
 → arr1 = {29, 34, 32, 19, 43, 12, 18, 23, 29, 12}
 arr2 = Same
 arr2 = {18, 26, 13, 30, 46, 25, 28, 30, 17, 20, 29, 34, 32, 19, 43, 12, 18, 23, 29, 12}

* In system verilog we have foreach Loop - Foreach
 In this foreach loop we need to mention directly elements
 array having size 10

Syntax: foreach (arr[i]) begin
 end

```

array.s → module tb;
int arr[];
initial begin
    arr = new[10];
    foreach (arr[i]) begin
        arr[i] = $urandom-range(10,20);
    end
    $display("arr = %p", arr);
end
endmodule

```

Q: Consider one array and assign random values b/w 1 to 5 and insert 10 at position 3 size of the array will be 5

Module tb;

int arr[5];

int arr1[5];

initial begin

arr = new[5];

arr1 = new[5];

foreach (arr[i]) begin

arr[i] = \$urandom-range(1, 5);

end

\$display ("before inserting arr = %p", arr);

arr1 = arr;

arr = new[6];

foreach (arr[i]) begin

if (i < 3) arr[i] = arr1[i];

else if (i == 3) arr[i] = 10;

else arr[i] = arr1[i-1];

end

\$display ("after inserting arr = %p", arr);

end

endmodule

∴ o/p: before inserting
arr = {3, 1, 3, 3, 2}

after inserting arr =
{3, 1, 3, 10, 3, 2}

Multidimensional array:

Multidimensional arrays means the array which contains more than one dimension

* How to represent multidimensional arrays

- static multidimensional arrays
- The size of the multidimensional arrays will be fixed

Syntax: <data-type> <array-name> [<size>] [<size>];

Ex: int arr[5][5];

module tb;

int arr[5][5];

initial begin

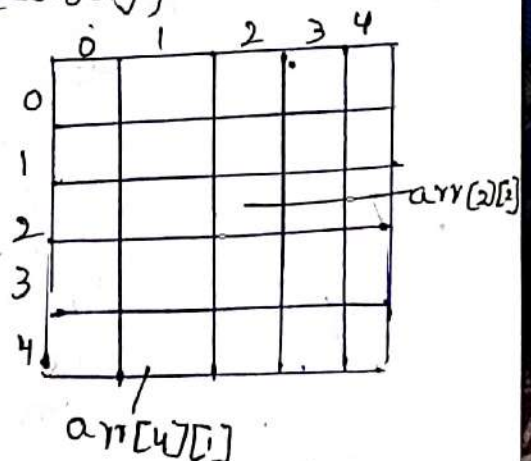
if (int i = 0; i < 5; i++) begin

foreach (arr[i]) begin

if (int j = 0; j < 5; j++) begin

arr[i][j] = \$urandom-range(5, 10);

end



```

$display ("arr[i] = %P", arr[i]);
end
// $display ("arr = %P", arr);
end
endmodule

```

o/p: arr = { { 5, 9, 7, 7, 9 },
 { 9, 9, 9, 10, 9 },
 { 5, 7, 8, 5, 8 },
 { 7, 5, 10, 10, 6 },
 { 10, 5, 8, 6, 8 } }

* Dynamic multi dimensional array

The size of the array will be allocated at run time

Syntax: <data-type> <array-name> [] []

```

// reg [3:0] [2:0] arr [5] [5];

```

```

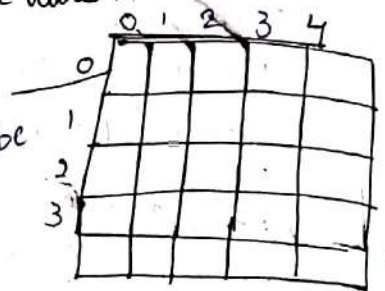
module tb;
  int arr [ ] [ ];
  arr <- newinitial begin
    arr = new [5];
    foreach (int i = 0; i < 5; i++) begin
      arr[i] = new [5];
      foreach (int j = 0; j < 5; j++) begin
        arr[i][j] = $urandom_range(10, 20);
      end
    end
    $display ("arr = %P", arr);
  end
endmodule

```

rows columns
 [3:0] [2:0] arr [5] [5]
 ↓ ↓
 How many: size of
 values the value

3 values

↓
 value will be
 0 to 15



o/p: arr = { { 17, 20, 8, 13, 15 },
 { 19, 13, 20, 16, 13 },
 { 19, 13, 14, 12, 20 },
 { 20, 15, 12, 15, 13 },
 { 11, 20, 13, 13, 19 } }

```

// reg [3:0] [2:0] arr [5] [5];

```

```

module tb;
  reg [2:0] [3:0] arr [5] [5];
  initial begin
    foreach (arr[i]) begin
      foreach (arr[i][j]) begin
        arr[i][j] = $random;
      end
    end
    $display ("arr = %P", arr);
  end
endmodule

```

o/p: 0 { { { } , { } , { } , { } , { } }

1

2

3

4

5

"req [3:0][2:0] arr[5][5];

no. of bits in array = $5 * 5 * 4 * 3$ bits

no. of bits in array[3] = $5 * 4 * 3$ bits

" " " array[3][2] = $4 * 3$ bits

" " " array[3][2][1] = 3 bits

" " " array[3][2][1][1] = 1 bit (0 or 1)

10/02/25.

User methods in dynamic arrays

Copy

Insert

compare

delete one element

Reverse

Compare:

Module tb;

int arr1[];

int arr2[];

int arr3[];

initial begin

arr1 = new[10];

arr2 = new[10];

arr3 = new[10];

foreach (arr1[i]) begin

arr1[i] = \$urandom_range(10,20);

arr2[i] = \$urandom_range(30,40);

end

arr3 = arr1;

\$display("arr1 = %p", arr1);

\$display("arr2 = %p", arr2);

\$display("arr3 = %p", arr3);

if (arr1 == arr2) \$display("both arr1 and arr2 are same");

else

\$display("both arr1 and arr2 are not same");

if (arr1 == arr3) \$display("both arr1 and arr3 are same");

else

\$display("both arr1 and arr3 are not same");

end

endmodule

Delete

Module tb;

int ar[];

int arr[];

int position;

initial begin

ar = new[10];

arr = new[9];

position = 5;

foreach (ar[i]) ar[i] = \$urandom_range(10,20);

foreach (arr[i]) begin

if (i < position) arr[i] = ar[i];

else

arr[i] = ar[i + 1];

end

arr[i] = ar

\$display("before deleting the element arr = %p", ar);

\$display("after deleting the element arr = %p", arr);

end

endmodule

o/p: before deleting: {17, 20, 18, 13, 5, 19, 13, 20, 16, 8}

after: {17, 20, 18, 13, 15, 13, 20, 16, 13}

Types of dynamic arrays.

① Dynamic array ② Queue ③ Associative array

① Queue: queue is also a dynamic array but in dynamic array we need to allocate the size by using new method

• But in queues no need to allocate size, it will adjust according to the requirement.

• In queue we have some extra methods.

Syntax: `<data-type> <queue-name> [$];`

`<data-type> <queue-name> [$:size];`

* Types of Queue: 2 → 1. Bounded 2. Unbounded Queue.

1. Bounded queue: the size of the queue is fixed.

Syntax: `<data-type> <queue-name> [$:size];`

Ex: `int intQ[$:19];`

number of locations of the queue is 20

|| Number of locations of bounded queue is "size+1"

2. Unbounded queue: the size of queue is not fixed

Syntax: `<data-type> <queue-name> [$];`

Ex: `int intQ[$], bit bitQ[$], etc...`

|| Bounded and unbounded queue

Module 4b;

`int intQ1[$:19];`

`int intQ2[$];`

`initial begin`

`for(int i=0; i<20; i++)`

`int Q1[i] = $urandom_range(10,20);`

`$display("intQ1 = %p", intQ1);`

`for(int i=0; i<10; i++)`

`int Q2[i] = $urandom_range(10,20);`

`$display("intQ2 = %p", intQ2);`

`$display("after modifying the sizes");`

`for(int i=0; i<25; i++)`

`int Q1[i] = $urandom_range(10,20);`

`$display("intQ1 = %p", intQ1);`

`for(int i=0; i<20; i++)`

`int Q2[i] = $urandom_range(10,20);`

`$display("intQ2 = %p", intQ2);`

`end`

`endmodule`

`o/p:`

→ out of bounded

Queue Operation would exceed
right index of 19, 20, 21, 22,

* Methods in queue

1. Size: it tells about size of the queue.

Syntax: Queue-name.size();

2. Insert: used to insert the element in a specified Index Location

Syntax: Queue-name.insert(i, value);

i means index and value means which value we want to insert

3. Delete: used to delete the entire queue

Syntax: Queue-name.delete();

if we want to delete in a specified index location

Syntax: Queue-name.delete(i);

4. push-back: it will push the element into the queue from backward direction

Syntax: Queue-name.push-back(value);

5. push-front: it will push the element into the queue in forward direction

Syntax: Queue-name.push-front(value);

6. pop-back: getting value from the queue in ^{back} forward direction

Syntax: value = Queue-name.pop-back();

7. pop-front: getting value from the queue forward direction

Syntax: value = Queue-name.pop-front();

8. Shuffle: It will shuffle all the elements in the queue

Syntax: Queue-name.shuffle();

9. Reverse: It will reverse the queue

Syntax: Queue-name.reverse();

Module tb;

int intQ[5];

int n, value;

initial begin

for (int i=0; i<20; i++) begin

intQ[i] = \$random % 40;

end

\$display("intQ = %p", intQ);

1. Size

2. Insert

3. Delete

4. push-back

5. push-front

6. pop-back

7. pop-front

8. Reverse

// size

n = intQ.size();

\$display("size of the queue = %0d", n);

// Insert

intQ.insert(3, 100);

\$display("after inserting element

intQ = %p", intQ);

//push back

```
intQ.push-back(50);
```

```
$display("after pushing the element in backside intQ = %p", intQ);
```

//push front

```
intQ.push-front(40);
```

```
$display("after pushing the element in front intQ = %p", intQ);
```

//Pop-back:

```
value = intQ.pop-back();
```

```
$display("Popped value = %d", value);
```

```
$display("after popping the element in backside intQ = %p", intQ);
```

//Pop-front

```
value = intQ.pop-front();
```

```
$display("Popped value = %d", value);
```

```
$display("after popping the element in frontside intQ = %p", intQ);
```

//Shuffle

```
$display("before shuffling intQ = %p", intQ);
```

```
intQ.shuffle();
```

```
$display("after shuffling intQ = %p", intQ);
```

//reverse

```
$display("before reversing intQ = %p", intQ);
```

```
intQ.reverse();
```

```
$display("after reversing intQ = %p", intQ);
```

//delete

```
$display("before deleting intQ = %p", intQ);
```

```
intQ.delete(18);
```

```
$display("after deleting intQ = %p", intQ);
```

```
intQ.delete();
```

```
$display("after deleting intQ = %p", intQ);
```

end

endmodule

Q: consider an integer queue and insert 10 elements and remove first 5 elements from the queue.

Implement FIFO, LIFO behaviour using Queue

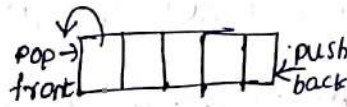
FIFO

LIFO

2 ways

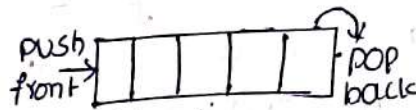
① push-back
pop-front

① push-back
pop-back



② push-front
pop-back

② push-front
pop-front



```
module tb;
int intQ[$];
int value;
initial begin
  for (int i=0; i<10; i++) begin
    value = $random % 100;
    intQ.push-back(value);
  end
  $display("intQ = %p", intQ);
  for (int i=0; i<5; i++) begin
    value = intQ.pop-front();
  end
  $display("intQ = %p", intQ);
end
endmodule
```

FIFO

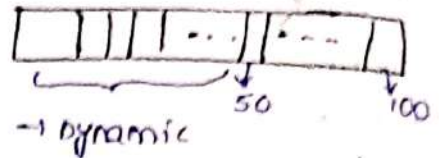
```
module tb;
int intQ[$];
int value;
initial begin
  $display("...pushing...");
  for (int i=0; i<10; i++) begin
    value = $urandom_range(10,50);
    // intQ.push-back(value);
    intQ.push-front(value);
  end
  $display("intQ = %p", intQ);
  $display("...popping...");
  for (int i=0; i<10; i++) begin
    // value = intQ.pop-front();
    value = intQ.pop-back();
    $display("intQ = %p", intQ);
  end
end
endmodule
```

LIFO

```
module tb;
int intQ[$];
int value;
initial begin
  $display("pushing");
  for (int i=0; i<10; i++) begin
    value = $urandom_range(10,50);
    intQ.push-back(value);
  end
  $display("intQ = %p", intQ);
  $display("...popping...");
  for (int i=0; i<10; i++) begin
    // value = intQ.pop-front();
    value = intQ.pop-back();
    $display("intQ = %p", intQ);
  end
end
endmodule
```

Associative array

we have of size 100 if i want access 50th location means



not i use queue or dynamic array

means first i need to access 0 to 49

locations after that i can access 50th location

- Because dynamic array and queues are continuous index access, to avoiding this issue we go with associative arrays.

Associative: Associative array means we can provide random value with random index

syntax: `<array-data-type> <array-name> [<index-data-type>];`

Ex: declare int array with int index

```
int arr[int];
```

declare int array with index as string

```
int arr[string];
```

default associative array index type is "int"

```
<array-data-type> <array-name> [int];
```

```
<array-data-type> <array-name> [*];
```

Declaration of associative array

11/02/25

```
<array-data-type> <array-name> [<index-data-type>];
```

Ex: `int arr[int];`

```
real arr[*];
```

```
reg arr[string], etc...
```

```
Module tb;
```

```
int arr[*];
```

```
int index;
```

```
initial begin
```

```
"for (int i=0; i<10; i++) begin
```

```
repeat (20) begin;
```

```
index = $urandom-range(10,50);
```

```
arr[index] = $urandom-range(100,200);
```

```
$display("arr[%d] = %d", index, arr[index];
```

```
end
```

```
$display("arr=%p", arr); end
```

```
endmodule
```

Q: Declare an associative array with array type int and index string now generate 10 random values b/w 100 to 200

Module 16;

int arr[string];

int value;

string index;

initial begin

repeat (10) begin

value = \$urandom-range(65, 97);

// index = string(value);

// \$sf_format(index, "%05s", value);

index = \$sf_format("%05s", value);

arr[index] = \$urandom-range(100, 200);

end

\$display("arr = %p", arr);

end

endmodule

Methods in associative array

1. Num: No. of elements in associative array
Syntax: array-name.num();
2. Exists: Index is exist or not
Syntax: array-name.exists(index);
3. First: Index of the first associative array element
Syntax: array-name.first(index);
4. Next: Next index of the associative array
Syntax: index = array-name.next();
5. Last: Last index of the associative array
Syntax: index = array-name.last();
6. Prev: previous index value of the associative array
Syntax: array-name.prev(index);
7. Delete: delete complete associative array or we can delete specific index value
Syntax: for deleting complete array
array-name.delete();
Syntax: specified index
array-name.delete(index);

array.3

Module +b;

int arr[+];

int index;

int n;

initial begin

repeat (10) begin

index = \$urandom_range(10,50);

arr[index] = \$urandom_range(100,200);

end

\$display("arr = %p", arr);

// num method

n = arr.num();

\$display("number of the array = %d", n);

// exist method

maxx.

if (arr.exists(29))

\$display("index is exists");

else \$display("index is not exist");

if (arr.exist(5))

\$display("index is exist");

else \$display("index is not exist");

// first method

void' (arr.first(index));

\$display("the first index and value arr[%d] = %d", index, arr[index]);

// next method

void' (arr.next(index));

\$display("next index and value arr[%d] = %d", index, arr[index]);

// Last method

void' (arr.last(index));

\$display("last index and value arr[%d] = %d", index, arr[index]);

// prev method

void' (arr.prev(index));

\$display("prev index and value arr[%d] = %d", index, arr[index]);

// delete method

arr.delete(index);

\$display("arr after deleting the value at index %d is = %p", index, arr);

```

arr.delete(i);
display("arr after deleting all the element = 'p', arr);
end
endmodule

```

Q: Consider an associative array with index will be int type and array also int type generate 100 random value in random indices range for values (100, 150) and index range (10, 50) after that consider 2 queues store the indices in one queue and value in another queue.

Operators in system verilog

1. arithmetic operators : +, -, *, /, %, **

output possibilities will be 0, 1, and x

2. Logical operators : &&, ||, ! → true or false

a	b	a & b	a b
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

&& means both statement need to be true then only the output will be 1 else 0

|| any one statement.

3. Unary operators : ~, |, ^, ~^, ~&, ~||

4.

The output possibilities will be 0, 1 or x

Logical:

```

Module tb;
  reg [3:0] a, b;
  reg c;
  initial begin
    a = 4'b0010;
    b = 4'b1010;
    c = (a & b);
    display("Logical and = 'b", c);
    a = 4'b0010;
    b = 4'b0000;
    c = (a & b);
    display("Logical and = 'b", c);
    a = 4'b0010;
    b = 4'b00xx;
    c = (a & b);
    display("Logical and = 'b", c);
    a = 4'b0010;
    b = 4'b00xx;
    c = (a & b);
    display("Logical and = 'b", c);
  end
endmodule

```

// Logical or

```

a = 4'b0010;
b = 4'b1010;
c = (a || b);
display("Logical or = 'b", c);
a = 4'b0010;
b = 4'b0000;
c = (a || b);
display("Logical or = 'b", c);
a = 4'b0010;
b = 4'b10xx;
c = (a || b);
display("Logical or = 'b", c);
a = 4'b0000;
b = 4'b00xx;
c = (a || b);
display("Logical or = 'b", c);
// Logical Not
a = 4'b0010;
c = (~a);
display("Logical Not = 'b", c);

```

end
endmodule

O/p: Logical and = 1, 0, 1, x
" or = 1, 1, 1, x
not = 0, 1, x

Unary

Module tb;

reg [4:0] a;

reg c;

initial begin

a = 5'b00101;

c = ~a, ~a, |a, ~a, ~(~a), ~(~a), ~(~a), ~(~a)

\$display("Unary and = ~.b", c); not, or, xor, xnor, nand, nor

end

endmodule

O/p: unary and = 0, or = 1, not = 0, xor = 0, xnor = 1, nand = 1, nor = 0

4. Bitwise operators: &, |, ^, ~, ~~, ~~, ~~ → 0, 1 and x

Bitwise:

module tb;

reg [4:0] a, b, c;

initial begin

a = 5'b0010;

b = 5'b1011;

c = a & b, | (a | b), ^ (a ^ b), ~ (a & b), ~ (a | b);

\$display("and a = ~.b, b = ~.b, c = ~.b", a, b, c); => or, xor, xnor, nand, nor

Unknown values

\$display("with unknown values");

a = 5'b0010;

b = 5'bxx0x1;

c = a & b; // all operations same

\$display("and a = ~.b b = ~.b c = ~.b", a, b, c);

end

endmodule

O/p: Unknown; and a = 00010, b = xx0x1, c = 000x0 as it is all o/p's

5. Shift operators: verilog left, right shift

system verilog ① right ② Left

① << --- Left shift

>> --- Right

<<< --- signed left

>>> --- signed Right shift

③ signed left ④ signed right shift

Shift operator
module tb;

```
int a, b, c, d, e, f, g;
initial begin
  a = 2; b = 30; c = -30;
  d = b << a;
  e = b >> a;
  f = c <<< a;
  g = c >>> a;
```

```
$display("d = %b", d);
$display("e = %b", e);
$display("f = %b", f);
$display("g = %b", g);
end
endmodule
```

O/P: d = 0... 1111000 - 32 bits
e = 0... 111
f = 111... 1001000
g = 111... 000

6. Relational operator: >, <, >=, <=, 0, 1, and x

By using this shift operators can perform multiplication and division
multiplication (<<)
division (>>)

Q: Consider 2 shift-numbers 55 and 42 and perform multiplication using shift operator

55, 42
42 → 1 0 1 0 1 0 → maximum 5 shifts
2⁵ 4 3 2 1 0 2⁵

55 (32 + 8 + 2)
2⁵ + 2³ + 2¹ (5 << 5) → 32 × 55
55 × 32 + 55 × 8 + 55 × 2 (55 << 3) + (55 << 1)
+ (55 << 5)

Module tb
reg [5:0] a; reg [5:0] b; reg [11:0] c;
initial begin
a = 55; b = 42;
c = ((a << 3) + (a << 1) + (a << 5));
\$display("c = %d", c);
end
endmodule
O/P: c = 2310

7. Conditional operator: Basing on the conditional it will work
Y = (condition) ? - : -;

Q: write 2x1 mux using conditional operator

y = (s) ? i1 : i0;

8. Concatenation operator: It will combine 2 or more data { }
Module tb;

```
reg [4:0] a; reg [9:0] b; reg [15:0] c;
initial begin
  a = 5'b10101;
  b = 10'b000000 - 10110;
  c = {a, b};
  $display("a = %b b = %b", a, b);
  $display("c = %b", c); end
endmodule
```

Ex: reg [5:0] a;
reg [9:0] b;
integer c;
reg [25:0] d;
a = 10101;
b = 0000010110
c = 'd100;
d = {a, b, c};

9. Replication : It will replicate the value { }

Syntax: { <replication-value> {value} };

a = 1010 replicate this a value 5 times

{5 {4'b1010}} or {5 {a}};

10. Equality :

Logical
case
wild card

Logical: ==, !=

check the condition are same or not → 0, 1 and x → bits

case: ==, !=

output possibilities 0 and 1

It will check x with x and z with z

wild card: ==?, !=?

output possibilities are 0, 1 and x

Equality:

module tb;

reg [4:0] a, b;

reg c;

initial begin

~~same~~ a = 5'b0101;

b = 5'b10101;

c = (a == b);

\$display("c = %0d", c);

end

endmodule

// i have 2 vectors

a = 5'b10101

b = 5'b11011

even position i don't want to compare

i don't want to compare odd positions

" " " " " particular position

c = (a == ?b);

LHS will be a

RHS " " b

If we mention x or z in RHS position it will select in LHS

a = 5'b10101;

b = 5'b x 0 x 1 z;

LHS part: In LHS if i mention (x or z) in consideration position (not neglecting position)

we will check basing on bit by bit if we got unknown means it give x or else 0

$a = 5'b1010x$; $b = 5'bxx0x11$; output as 0

$a = 5'b1011x$; $b = 5'bxx0x11$; output as x

wild card;

```
module tb;
```

```
reg[4:0] a,b;
```

```
reg c;
```

```
initial begin
```

```
a = 5'b10111;
```

```
b = 5'bxx0x11;
```

```
c = (a == b);
```

```
$display("a = %b b = %b c = %b", a, b, c);
```

```
end
```

```
endmodule
```

o/p: a = 10111, b = xx0x11, c = 1

* Set membership operator;

13/02/25.

In membership operator we use "inside" keyword and we use {}

Inside these {} we can value or range

{value1, value2, value3 --- } // for values

{ [lower-value: higher-value] };

Ex: inside { [10:20], 5, 50, 55 };

Set-mem.v:

```
module tb;
```

```
int a;
```

```
initial begin
```

```
repeat (5) begin
```

```
a = $urandom-range(10, 50);
```

```
if (a inside { [10:20], 29, 34 })
```

```
$display("a is member a = %d", a);
```

```
else $display("a is not a member a = %d", a);
```

```
end
```

```
end
```

```
endmodule
```

o/p: a is member a = 29

" = 18

" = 34

a is not a member a = 2

" = 32

Limitation: for range we need to pass [lower-value: higher-value]

Streaming operators:

>> ... right streaming operator

LL - - Left: streaming opened.

It will stream the data into a queue

if i use `>>` means my last will stored in queue other index

n n n << n n -first n n n n n

Module -16;

```
1 byte unsigned a, b, c;
```

```
int unsigned a[10];
```

```
byte unsigned byte q [N];
```

initial begin

$$a=100; \quad b=150; \quad c=200;$$

```
byte q = >> byte {a,b,c};
```

11 byte Q = { 4 byte { a, b, c } }

```
$display ("byte  $\varphi$  =  $\gamma \cdot p$ ", byte $\varphi$ );
```

end

```
endmodule
```

concatenation:

reg [5:0] a;

```
reg[3:0] b;
```

```
int c;
```

```
reg [40:0] d;
```

$$d = \{a, b, c\};$$
$$d = \{c, b, a\},$$

int $\rightarrow$ 32 bit

representing 100 we require 8 bits

→ 00000 - - -

$\rightarrow 0, 0, 0, 100$

o/p: byte q = {100, 150, 200}

by-leaf: 300, 50, 100

$$HyICR = \{200, 900, 150, 800, 700, \\ 50, 60, 100, 0, 0, 0, 150, 800\}$$

* OOPS: object oriented programming language

in oops we will performing by object

- what is ment by object: instance of the class
- Instance means: imaginary reference of the class

* static and dynamic: during compile time

Ex: vectors, static, arrays, variable declarations, module instantiations.

* dynamic: run time

Ex: dynamic arrays, class etc...

* In static if i create instance means

In static if i create instance means it will create memory for the instance

* In dynamic if i create instance means:

In dynamic if i can instance means it will just create the imaginary reference

- allocate memory means it is an object

Class sample;

endclass

Module +b;

sample s; --- > //imaginary reference

initial begin

s = new(); //object

end

endmodule

Module:

- port declarations

- parameter
- functions and task
- continuous assignments
- Built in gates. → assign
- and, or, not ...
- procedural blocks
- initial and always
- procedural statements
- loops
- for, loop, while loop,
- repeat loop,
- forever loop
- conditional statements
- if, else, else if
- case statements
- case, casez, case2
- Blocking, Non blocking
- fork join
- task and function call
- operators
- system task and system functions
- envvar
- instantiations

In class → properties

① Properties: we have 2 types

1. object properties
2. non-object properties

② Methods: tasks and functions

③ Constraints: Limitations of the signals

class means:

- class is a userdefined data type
- class consists of properties, methods and constraints
- class is in dynamic in nature so to memory allocation we need to use new keyword.

Syntax: class class-name;

|| properties

|| methods

|| constraints

endclass

① Properties: These are signals of the class

1. Object Oriented property: The properties which are having the involvement of object is known as object property.

Ex: sample s;

2. Non-object properties: The properties which doesn't having the involvement of object is known as non-object property.

Ex: int a, bit b, etc---

② Methods: task and function

③ Constraints: These are the limitations of the properties

Syntax: constraint constraint-name {
constraint logic }

* Randomize: It is a predefined method it is used to generate a random values which are properties mention with rand or randc keywords.

* Rand: It will generate the values in random manner in rand repetition will be happen

* Randc: It will generate the values in cyclic manner Repetation will be happen after completion of all the all the possibilities

```

class sv
class sample;
    rand int a; //property
//method
    function void print();
        $display("a=%0d", a);
    endfunction
//constraints a-c
    a > 40;
    a < 50; }
endclass

```

```

module tb;
    sample s;
    initial begin
        s = new();
        repeat (10) begin
            s.randomize();
            s.print();
        end
    end
endmodule

```

Q: Note: If i allocate memory inside the repeat loop my randc will work as rand only.

Q: declare a class and in that class 1 sum method and consider a tb and declare 3 variables and assign the sum value to the 3rd variable by using above sum method

```

class sample;
    function int sum (input int a, b; output int y);
        sum = a + b;
    endfunction
endclass

```

```

module tb;
    int a, b, c;
    sample s;
    initial begin
        s = new();
        a = 10;
        b = 20;
        c = s.sum(a, b);
    end
endmodule

```

```

    $display("a=%0d b=%0d c=%0d", a, b, c);
end

```

```

Rand.sv
class sample;
    rand bit [1:0] a;
    rand bit [2:0] b;
    function void print();
        $display("a=%0d", a);
        // $display("b=%0d", b);
    endfunction
endclass

```

```

module tb;
    sample s;
    initial begin
        s = new();
        repeat (10) begin
            // s = new();
            s.randomize();
            s.print();
        end
    end
endmodule

```

∴ class in dynamic nature we have to allocate allocate memory
∴ create instance, not allocating memory

class mem.sv

class sample;

endclass

module tb;

sample s;

initial begin

if(!s) \$display("memory is not allocated");

else \$display("memory is allocated");

s=new();

if(!s) \$display("memory is not allocated");

else \$display("memory is allocated");

end

endmodule

* In verilog for random value generation we use 'rand' randomization keywords → \$random, \$urandom, \$urandom-range

* In class can we use this keyword or not
we can use those keywords in classes also

Random.sv

class sample; class random;

// rand int a,b;

function void print();

\$display("a=%0d b=%0d",a,b);

endfunction

endclass

module tb;

random r;

initial begin

r=new();

r.a=\$urandom-range(10,20);

r.b=\$urandom-range(30,40);

r.print();

end

endmodule

rand-method.sv

class sample;

rand a;

function void pre-randomize();

\$display("pre-randomization will be happen");

endfunction

// randomize method

* It is a callback method
whenever we call this randomize
function it will call it pre
and post function

// pre-randomize();

// randomize();

// post-randomize();

function void post-randomize();

\$display("post-randomization
will be happen");

a=100;

endfunction

```
Module tb;
sample s;
initial begin
    s=new();
    s.randomize();
    s.print();
end
```

Mode: It is used to disable or enable the mode for properties.

```
11 for enable the mode  
property-name.rend-mode(1);
```

```
11 for disable the mode  
12 constraint_name.constraint_model();
```

TEST-BENCH

ENV

SBD

AGENT

COV

MONI

BEM

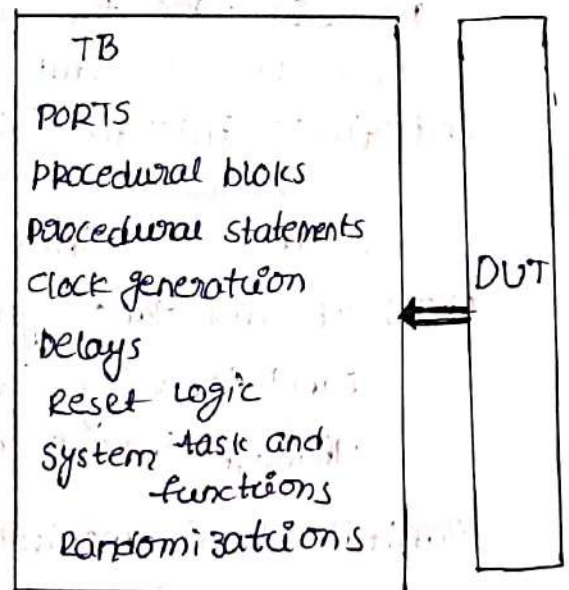
DUT

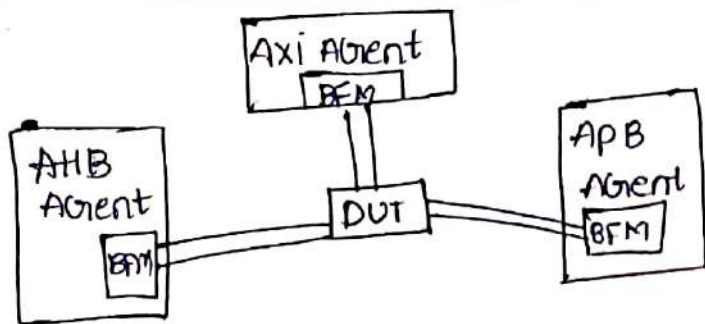
Interface

```
class Sample;
    rand int a,b;
function void print();
    display("a=%d, b=%d",
            a,b);
endfunction
endclass
```

g/p: $a = 772105924$, $b = 0$

19/09/25,,





Types of connections: 3 types of connections

1. class to class connection:
for class to class connection "mailbox" used
2. class to Module connection:
class to module connection "interface" used
3. Module to module connection:
By using "instance" we can connect one module to other module

Testbench architecture

DUT

ENV

SBD

AGENT

GEN

BFM

MON

COV

Generator: It will generate the transactions (tx) and it will send tx's to bfm through mailbox

Bfm (Bus functional model) driver:

It will take the tx's from mailbox and it will send those tx's to dut through interface

Interface: Interface is used to connect class and module
It will take tx's from bfm and it will convert those tx's into signals, and it will send those signals to dut.

DUT: It will take signals from interface and it will perform functionality and it will send the outputs to bfm through interface

Monitor: It will monitor the transfer's b/w bfm and dut and it will collect all the signals from interface and it will convert those signals into tx's and it will send those tx's to coverage and scoreboard through mailboxes.

Coverage: It will take tx's from mailbox
It will check about the signals are covered or not

Score board: It will take tx's from mailbox and it will perform the checking by using matches and mismatches

Agent: Gen, bfm, mon and cov will be available in agent

Environment: In env we have agent and scoreboard

Methods: Two types of methods we have

1. predefined: 1. NewC();
2. RandomizeC();

2. Userdefined: 1. print
2. copy
3. compare
4. pack
5. Unpack method

* components means class: Gen, bfm, cov, mon, sbd, agent, env

* Gen want print method and copy method:

we need to create print method and copy methods in gen

* Bfm want compare method and pack method:

we need to create compare method and pack methods in bfm

* Mon want ~~compare~~ print method, pack method and bfm unpack method

1. we need to create print method, pack method and unpack again

Extra work for us

2. we can call that print in gen and pack method in bfm
memory wastage will be happen

3. we can consider another class in that class we will write all the common methods, common properties and common constraints.

So that i will create memory for that particular class and i will use that properties, methods and constraints

→ That class is known as tx class

Transaction class: In tx class what ever the common properties, methods and constraints we need to mention.

o/p: apb-ix

pwwrite = 1

pwdata = 136

prdata = 0

paddr = 8

pser = 0001

perr = 0

Mailbox: Mail box is used to communicate between two classes
Used to transfer the data from one class to another class

Types of mailboxes: Mailbox will be classified in 2 ways

1. Basing on the types

2 types: Generic mailbox
parameterized mailbox

2. Basing on the size

2 types: Bounded mailbox
Unbounded mailbox

* Basing on types:

1. Generic mailbox: In Generic mailbox it will send any type of data
Syntax: mailbox mailbox-name;

2. parameterized mailbox: parameterized mailbox will send only particular parameter type of data

Syntax: mailbox#(parameter) mailbox-name;

* Basing on the size:

2 types: 1. Bounded mailbox:

The size of the mailbox will be fixed

Syntax: mailbox mailbox-name = new(5);

2. Unbounded mailbox: The size of the mailbox is not fixed

Syntax: mailbox mailbox-name = new();

Methods in mailbox:

1. New: This method is used to allocate memory for the mailbox
Syntax: mailbox-name = new();

2. PUT: This method is used to put the data into the mailbox
This method having blocking nature
Syntax: mailbox-name.put(<data>);

3. Get: This method is used to get the data from the mailbox
in get method we have blocking nature

Syntax: mailbox-name.get(<variable>) --> which variable we need to get the data

4. tryput: It is used to put the data into the mailbox
it doesn't having the blocking nature

Syntax: mailbox-name.tryput(<data>);

5. tryget: It is used to get the data from the mailbox
It doesn't having blocking nature

Syntax: mailbox-name.tryget(<variable>) -- which variable we need to get the data

6. Num: It will tell about number of elements present in
The mailbox

Syntax: n = mailbox-name.num();

7. peek: It will tell about the first element of the mailbox

Syntax: variable = mailbox-name.peak();

Mailbox.SV:

Module tb;

mailbox mbox = new();

int value;

int n, x;

initial begin

// put method

for(int i=0; i<10; i++) begin

value = \$orandom_range(10, 50);

mbox.put(value);

end

\$display("mbox = %p", mbox);

// Num method

n = mbox.num();

\$display("n = %d", n);

// Get method

for(int i=0; i<10; i++) begin

mbox.get(value);

\$display("value = %d", value);

end

// tryput method

for(int i=0; i<10; i++) begin

value = \$orandom_range(10, 50);

mbox.tryput(value);

end

\$display("mbox = %p", mbox);

// tryget method

for(int i=0; i<10; i++) begin

mbox.tryget(value);

\$display("value = %d", value);

end

// peek method

x = mbox.peak();

\$display("peek value x = %d", x);

end

endmodule

Lab: we have 100 computers, these computers are connected with LAN (Local area network) in that LAN we have switches, each switch will connect to the corresponding computer. Now if I want send a movie file with 600MB

preamble - 56

sof - start of frame -> 8 bits

sa - 48 bits

da - 48 bits

length - 16 bits

payload: it will be a queue and size of the queue will be depends on length

crc - 32 bits

length value will be 42 to 1500 bytes

eth-pkt.sv

```
class eth_pkt;
```

```
rand bit [55:0] preamble;
```

```
rand bit [7:0] sof;
```

```
rand bit [47:0] sa;
```

```
rand bit [47:0] da;
```

```
rand bit [15:0] len;
```

```
rand byte payload[8];
```

```
bit [31:0] crc;
```

```
function void print(string str);
```

```
$display("...:s- ", str);
```

```
$display("preamble = %b", preamble);
```

```
    sof = %b
```

```
    sa = %d
```

```
    da = %d
```

```
    len = %d
```

```
    payload = %p
```

```
    crc = %b
```

```
endfunction
```

```
constraint len_c {
```

```
    len inside {42:1500};
```

```
    len inside {10:20};
```

```
    payload.size() == len; } endclass
```

```
module tb;
```

```
eth_pkt pkt;
```

```
initial begin
```

```
    pkt = newC();
```

```
    pkt.randomize();
```

```
    pkt.print("eth_pkt");
```

```
end
```

```
endmodule
```

```
o/p: preamble = 00110011101...  
----->56 bits
```

```
sof = 110111111 ->8 bits
```

```
sa = 34709153859871
```

```
da = 41631366154925
```

```
len = 14
```

```
payload = {119, -42, 70, 75, 95,
```

```
    -50, 96, 114, 68, 32, 65,
```

```
    -12, 123, 123}
```

```
crc = 0000 ->32 bits.
```

~~eth-pkt~~ → program upto en

gen: eth-gen.sv
mailbox gen2bfm = new();

```
string testCase;  
class eth-gen;  
    eth-pkt pkt;  
    task run();  
    testCase = "5-pkt";  
case(testCase)  
    "5-pkt": begin  
        pkt = new();  
        pkt.randomize();  
        gen2bfm.put(pkt);  
        pkt.print("gen-prints");  
    end  
    "5-pkt": begin  
        repeat(5) begin  
            pkt = new();  
            pkt.randomize();  
            gen2bfm.put(pkt);  
            pkt.print("gen-prints");  
        end  
    end  
endcase  
endtask  
endclass
```

```
'include "eth-pkt.sv"  
'include "eth-gen.sv"  
'include "eth-bfm.sv"  
'include "eth-agent.sv"  
module tb;  
    eth-agent agent;  
    initial begin  
        agent = new();  
        agent.run();  
    end  
endmodule
```

Bfm: class eth-bfm; eth-bfm.sv

```
eth-pkt pkt;  
task run;  
forever begin  
    pkt = new();  
    gen2bfm.get(pkt);  
    pkt.print("bfm-prints");  
end  
endtask  
endclass
```

Agent: class eth-agent; eth-agent.sv

```
eth-gen gen;  
eth-bfm bfm;  
task run();  
forever begin  
    gen = new();  
    bfm = new();  
    fork  
        gen.run();  
        bfm.run();  
    join  
end  
endtask  
endclass
```

o/p
→ gen-prints

bfm-prints → 5 pkt → 5 times

Static - sv

```
class sample;  
    static int a;  
    static function void print();  
    $display("a = %d", a);  
endfunction  
endclass  
  
module tb;  
    // sample s;  
    initial begin  
        // s = new();  
        // s.a = 100;  
        // s.print();  
        sample::a = 10; // → scope  
        sample::print(); // resolution oper  
    end  
endmodule
```

Static:

static is a variable which is used to access the variable or methods without creating memory for the component

← Q: How to access those static variables and methods by using class name and scope resolution operator (::)

scope resolution operator (::)

It is used to access the static properties and methods

Difference b/w xmr and scope resolution operator

- In xmr we can access the variable by creating memory
- In scope resolution operator we can access static variable without creating memory for the class

Scope: tells about the range

Types of scope: 1. Global

2. Class/module

3. Task or function

Global scope: we can declare properties or methods outside the class or module

- Everyone can access the global scope properties or methods
- For accessing global variables we can use "\$Unit" and "scope resolution operator"

Global - sv

```
int a;  
function void print();  
    $display("a = %d", a);  
endfunction
```

module tb;

```
    initial begin  
        $Unit::a = 100;  
        $Unit::print();  
    end  
endmodule
```

- 11 Class scope means: whatever the properties or methods declared in the class those all are comes under to class scope.
- Module scope: whatever the properties or methods we declare in the module those are comes under module scope.
- Task scope: whatever the properties we declare inside the task those all are come under task scope.
- Function scope: whenever the properties we declare inside the function those all come under function scope.

\$unit: it is used for accessing the global properties and methods.

\$root: it is used to access the module or class without have instance.

→ by using

\$root, class or module name

Root.sv:

```
class s1;
```

```
  int a;
```

```
endclass
```

```
module s2;
```

```
  s1 s1_inst;
```

```
  initial begin
```

```
    s1_inst = new();
```

```
  end
```

```
endmodule
```

```
module s3;
```

```
  s2 s2_inst;
```

```
endmodule
```

```
module s4;
```

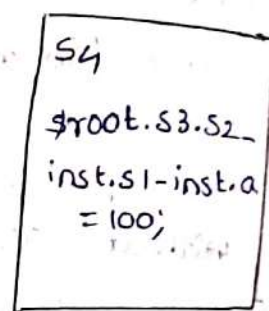
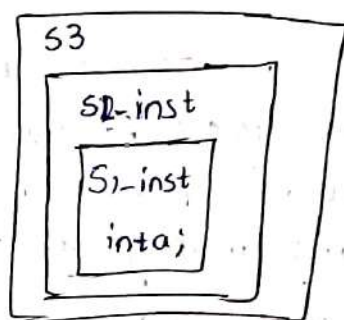
```
  initial begin
```

```
    $root.s3.s2_inst.s1_inst.a = 100;
```

```
    $display("a = %0d", $root.s3.s2_inst.s1_inst.a);
```

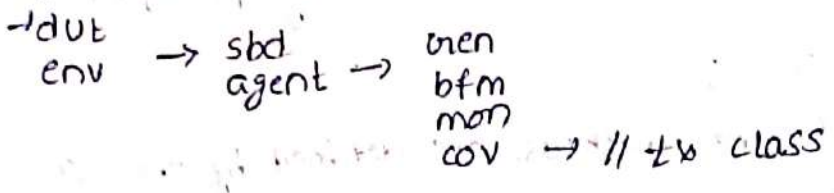
```
  end
```

```
endmodule
```



Top to bottom approach
Q: how to write system verilog test bench template

// Top module



Top.sv ①

```

Module tb;
  reg clk, res;

  // dut instantiation
  // env instantiation
  env e;

  // generate the clock
  // we need to apply the rest
  initial begin
    e = new();
    e.run();
  end
endmodule
  
```

② Env.sv

```

class env;
  agent a;
  sbd s;

  task run();
    a = new();
    s = new();
    fork
      a.run();
      s.run();
    join
  endtask
endclass
  
```

Agent.sv - ③

```

class agent:
  gen g;
  bfm b;
  mon m;
  cov c;
  task run();
    g = new();
    b = new();
    m = new();
    c = new();
    join
  endtask
endclass
  
```

④ Sbd.sv

```

class sbd;
  trans tx;
  task run();
    tx = new();
  endtask
endclass
  
```

⑤ Gen.sv

```

class gen;
  trans tx;
  task run();
  endtask
endclass
  
```

⑥ bfm.sv

```

class bfm;
  trans tx;
  task run();
  endtask
endclass
  
```

⑦ mon.sv

```

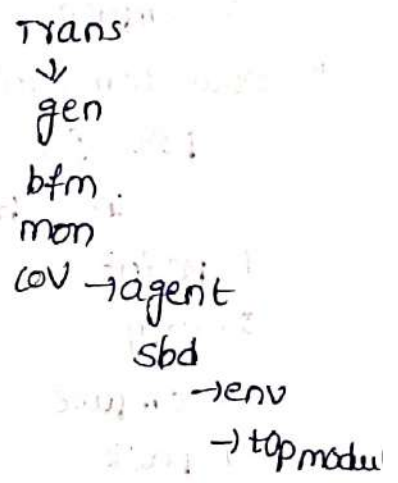
class mon;
  trans tx;
  task run();
  endtask
endclass
  
```

⑧ cov.sv

```

class cov;
  trans tx;
  task run();
  endtask
endclass
  
```

Bottom up approach



Class trans;
// common properties
// common methods
// common constraints
endclass

Interface
Interface intf;
endinterface

Methods in class: 2 types of methods

1. pre defined
2. user defined method

3. pre defined method: The method which are already pre-exist

Ex: 1. new();
2. Randomize();

2. User defined method: The methods are defined by the user

* Parallel communication:

We have dedicated ports for all the signals

Like for address we are having address pin "addrs"

for data we have data pins "wdata" and "rdata"

for control signals also having dedicated pin "valid", "ready"

* so we consider only these 3 main important methods

1. print

2. copy

3. compare

* Serial communication: In serial communication we don't dedicated ports for address, data and control signals

We have 5 methods

1. print

2. copy

3. compare

4. pack

5. Unpack

• data will be send through the single channel so we need to consider pack and unpack methods to send the data

In pre-defined method: New is a constructor, it is used for allocating memory and in the same way it is also used as a method

// function sum(int a, int output, int c);

c = a + b;

endfunction

// function call

sum(5, 10, c);

Serial communication: whatever the data will be there we need pack and send in the transmitter side
In receiver side we will unpack and send

```

Take eth_pkt.sv → new value
new-value.sv → class eth_pkt;
function new();
    preamble = 32;
    sof = 46;
    sa = 60;
    da = 55;
    len = 18;
    crc = 0;
endfunction

```

```

→ point
constraint
module tb;
    eth_pkt pkt;
    initial begin
        pkt = new();
        // pkt = randomize();
        pkt.print("eth_pkt");
    end
endmodule

```

Randomize() method:

It is used to generate random values for the properties which are mentioned with rand and randc.

It is also known as a call back method.

if i call randomize method,

first it will call pre-randomize()
randomize()

at the end it will call post-randomize()

with the help of pre-randomize and post-randomize we will get complete control on the signals.

assertions: assertions are used for checking purpose.

assertions are also known as pin point checkers.

Assertions 2 types → immediate assertion
concurrent assertion

```

New argu.sv
class eth_pkt.sv
function new(input bit [55:0] preamble_t,
    bit [7:0] sof_t, bit [47:0] sa_t, bit [4:0] da_t,
    bit [15:0] len_t, bit [31:0] crc_t);
    preamble = preamble_t;
    sof = sof_t;
    sa = sa_t;
    da = da_t;
    len = len_t;
    crc = crc_t;
endfunction
print
constraint
module tb;
    eth_pkt pkt;
    initial begin
        pkt = new(30, 40, 50, 60, 70, 0);
        // pkt = randomize();
        pkt.print("eth_pkt");
    end
endmodule

```

Immediate assertion: check will be happened at that point of time
Concurrent assertion: check will be happen at a specified time

Rand.sv

eth-pkt.sv

print

function void pre-randomize();

sof.rand-mode(0);

sa.rand-mode(0);

da.rand-mode(0);

P-C.constraint-mode(0);

endfunction

function void post-randomize();

sof=10;

sa=20;

da=20;

endfunction

constraint len-c{

len inside {10:20};

Payload.size() == len;

}

constraint p-c{

Preamble inside {10:20};

}

endclass

module tb;

eth-pkt pkt;

initial begin

pkt=new();

assert(pkt.randomize());

pkt.print("eth-pkt");

end

endmodule

assert.sv

class sample;

int a;

endclass

module tb;

sample s;

initial begin

s=new();

s.a=10;

assert(s.a!=0)begin

\$display("a value is greater than

end

else begin

\$display("a value is less than 0");

end

s.a=0;

assert(s.a!=0)begin

\$display("a value is greater than 0");

end

|| else begin

|| \$display("a value is less than 0");

|| end

end

endmodule

22/02/25. - 14

11 how to create user defined data types with existed data types

for creating user defined data type we need to consider "typedef"
syntax: typedef existed data type <size> <user-data-type-name>

Ex: i want to create nibble data

nibble means 4 bits

```
typedef bit[3:0] nibble;
```

user.sv

```
typedef bit[3:0] nibble;
```

```
module tb;
```

```
    nibble a;
```

```
    initial begin
```

```
        a = $random;
```

```
        $display("a = %b", a);
```

```
    end
```

```
endmodule
```

User defined methods:

1. print
2. copy
3. compare

4. pack method

we need to consider a queue
and pack the data into that queue

→ for packing and unpacking we need to use the streaming operators

eth-pkt.sv

```
typedef byte byteq[$];
```

eth-pkt.sv

Print

```
function eth_pkt copy();
```

```
    eth_pkt temp;
```

```
    temp = new();
```

```
    temp.preamble = preamble;
```

```
    temp.sof = sof;
```

```
    temp.sa = sa;
```

```
    temp.da = da;
```

```
// create a integer array with  
name arr;
```

```
typedef integer arr[10];
```

```
// create a byte queue with  
name byteq
```

```
typedef byte byteq[$];
```

user1.sv

```
typedef integer arr[10];
```

```
module tb; arr b;
```

```
    initial begin
```

```
        for(int i=0; i<10; i++) begin
```

```
            b[i] = $urandom_range(10,20);
```

```
        end
```

```
        $display("b = %p", b);
```

```
    end
```

```
endmodule
```

```
temp.len = len;
```

```
temp.payload = payload;
```

```
temp.crc = crc;
```

```
return temp;
```

```
endfunction
```

Compare

```
function bit compare (eth-pkt temp);
```

```
if (temp.preamble == preamble KK
```

```
temp.sof == sof KK
```

```
temp.sa == sa KK
```

```
temp.da == da KK
```

```
temp.len == len KK
```

```
temp.payload == payload KK
```

```
temp.crc == crc) return 1; else return 0;
```

```
endfunction
```

```
// pack method
```

```
function bytearray pack ();
```

```
bytearray que;
```

```
que = {LL byte {preamble, sof, sa, da, len, payload, crc}};
```

```
return que;
```

```
endfunction
```

```
// unpack method
```

```
function eth-pkt unpack (bytearray temp);
```

```
eth-pkt ep;
```

```
ep = new();
```

```
{LL byte {ep.preamble, ep.sof, ep.sa, ep.da, ep.len, ep.payload,
```

```
ep.crc}} = temp;
```

```
return ep;
```

```
endfunction
```

```
constraint len < {
```

```
// len inside { [42:1500] };
```

```
len inside { [10:20] };
```

```
payload.size() == len;
```

```
endclass
```

```
Module tb;
```

```
eth-pkt pkt1, pkt2, pkt3;
```

```
bytearray a;
```

```
initial begin
```

```
pkt1 = new();
```

```

pkt3 = new();
assert (pkt1.randomize());
// pkt2.randomize();
pkt1.print("pkt1");
pkt2 = pkt1.copy(); // copy
pkt2.print("pkt2");
if (pkt1.compare(pkt2)) $display("pkts are same"); // compare
else $display("pkts are not same");
a = pkt1.pack(); // pack
$display("a = %p", a);
pkt3 = pkt1.unpack(a); // unpack
pkt3.print("pkt-3");
if ("pkt1.compare(pkt3)") $display("pkts are same");
else $display("pkts are not same");
end
endmodule

```

OOPS: oops will supports

1. encapsulation
2. inheritance
3. polymorphism
4. abstraction

Encapsulation: keeping all the methods, properties and constraints into a single entity

• It will provide the data hiding

Inheritance: whatever the properties, methods and constraints contained by the parent those things will be inherited to the child

polymorphism: poly means many
morphism changing the forms

basing on the requirement output changes its form

abstraction: In abstraction we will consider abstract class

- → In this abstract class we have only virtual methods and virtual properties.

typedef byte byteQ[4];

eth_pkt_t sv

Print sv

function void ^{copy} copy (input eth_pkt_t temp1, output eth_pkt_t temp2;

temp2 = new();

temp2.preamble = temp1.preamble;

temp2.sof = temp1.sof;

temp2.sa = temp1.sa;

temp2.da = temp1.da;

temp2.len = temp1.len;

temp2.payload = temp1.payload;

temp2.crc = temp1.crc;

compare endfunction

function void compare (input eth_pkt_t temp, output bit value); ^{compare}

if (temp.preamble == preamble &&

temp.sof == sof &&

temp.sa == sa &&

temp.da == da &&

temp.len == len &&

temp.payload == payload &&

temp.crc == crc) value = 1;

else value = 0;

endfunction

pack method

function void pack (input eth_pkt_t ep, output byteQ temp);

temp = {<< byte {ep.preamble, ep.sof, ep.sa, ep.da, ep.len, ep.payload,

ep.crc}};

endfunction

unpack

function void unpack (input byteQ temp, output eth_pkt_t ep);

ep = new();

{<< byte {ep.preamble, ep.sof, ep.sa, ep.da, ep.len, ep.payload,

ep.crc}} = temp;

endfunction

constraint len < { len inside {10:20}};

payload.size() == len; } endclass

User defined methods:

1. print: function void print();
syn: // mentioned the display contents
endfunction

compare method: 2 pkts compared

2. function bit compare(input eth-pkt pkt);
endfunction

copy: whatever the fields available
in the eth-pkt copied the
data into the other pkt

syn: function eth-pkt copy();
// copy the pkt field
endfunction

Pack: Available data need to pack the data into queue

- These methods are used to transfer the data in serial communication
- In serial communication → single channel to transfer address, data, control signals

• used streaming operators → ^{syn:} function byteq pack();
// inside pack the signals
endfunction

unpack: unpack the data from queue

streaming operator for unpacking the data

syn: function eth-pkt unpack(input byteq byteq);
// unpack the data
endfunction

Note: Need to use the same streaming operator for packing and unpack;

// function with only input arguments

// function with only output arguments

// function with both input and output arguments

1. Encapsulation: properties, methods and constraints all are
group into a single entity.

Help of encapsulation we can hide the data

Data Hiding: properties or methods with a keywords

1. local 2. protected 3. public

Local: only parent class or base class will access the properties
or methods or constraints (Board of directors)

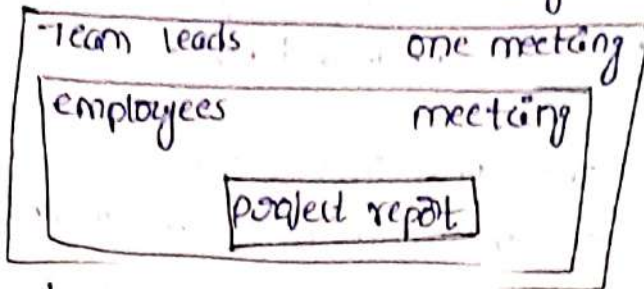
Protected: only parents and child will access those properties
or constraints

Public: everyone can access

company

Board of directors

company main meeting



data-hiding.sv

```
class sample;
  local int a;
  protected int b;
  int c;
  function void print();
    $display("a=%0.0d", a);
    $display("b=%0.0d", b);
    $display("c=%0.0d", c);
  endfunction
endclass
```

class child extends sample;

function new();

// super.a = 100;

super.b = 200;

super.c = 300;

endfunction

function void print();

super.print();

endfunction

endclass

Module tb;

// sample s;

child ch;

initial begin

// s = new();

ch = new();

// s.a = 100; // s.c = 300;

// s.b = 200; // s.print();

ch.print(); end

endmodule

parent

properties

methods

constraints

child

child we access whatever the properties, methods and constraints

data-overriding.sv

class sample;

local int a;

protected int b; int c;

function void print();

\$display("a=%0.0d", a);

\$display("b=%0.0d", b);

\$display("c=%0.0d", c);

endfunction

function void set(input int
a-t, b-t);

a = a-t;

b = b-t;

endfunction

endclass

Module tb;

sample s;

initial begin

s = new();

// s.a = 100; // s.b = 200;

s.c = 300;

s.set(100, 200);

s.print(); end

endmodule

∴ data overriding not possible in inheritance

* for accessing local and protected variables inside the module means we need to consider a function to override the local and protected variables.

• for a good programming practice we need to consider the properties and methods as protected.

Inheritance: Inheritance means the properties and methods and constraints are inherited from base to derived class

// how to write inheritance: we use a keyword called "Extends".

Syntax: class child-class-name extends parent-class-name;

// need to mention about the child class properties.

endclass

Inheritance:

```
class parent;
  int a;
  int b;
  function void print();
    $display("a=%d", a);
    $display("b=%d", b);
  endfunction
endclass

class child extends parent;
  function void print();
    super.print();
  endfunction
endclass
```

module tb;

parent p;

child ch;

initial begin

p=new();

p.a = 100;

p.b = 200;

p.print();

ch=new();

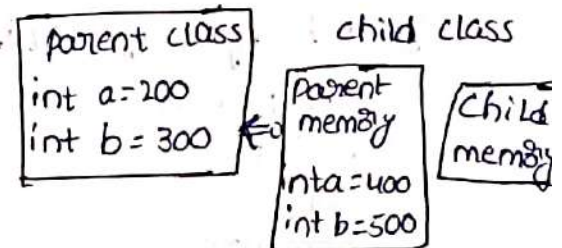
ch.a = 400;

ch.b = 500;

ch.print();

p.print();

end



* This: tells about This class

Super: tells about parent class;

: This-Super-SV:

class parent;

inta;

function new();

a=100;

endfunction

endclass

class child extends parent;

inta;

function new();

a=200;

endfunction

function void print();

\$display("parent value=%d", super.a);

\$display("child value=%d", this.a);

endfunction

endclass

↓

```

Module tb;
  child ch;
  initial begin
    ch = new();
    ch.print();
  end
endmodule

```

* an eth-pkt class, consider 3 child class
 1. good-pkt 2. bad-pkt 3. ill-pkt.
 consider count signals
 for good-pkt consider good-count
 " bad - " " bad - "
 " ill - " " ill - "

Q: write a test cases for good-pkt, bad-pkt and ill-pkt

consider a test variable → 2 bit variable

00 → good-pkt 01 → bad-pkt 10 → ill-pkt

default \$display("case is not possible");

Inheritance:

25/02/25 - 16

typedef byte byteQ[4];

```

eth-pkt sv
  print
  copy
  compare
  pack
  unpack
  constraint
  Module → (comparision)
  for (int i=0; i<10; i++) begin
    pkt[i] = new();
  end
  assert (pkt[i].randomize());
  pkt[i].print("eth-pkt");
end
endmodule

```

```

Bad-pkt.sv
class bad-pkt extends eth-pkt
  static int bad-count;
  function new();
    bad-count++;
  endfunction
  function void print(string
    str = "bad-pkt");
    $display("---%05s---", str);
    super.print();
    $display("bad-count = %0d",
      bad-count);
  endfunction
endclass

```

↓

```

Good-pkt.sv
class good-pkt extends eth-pkt;
  static int good-count;
  function new();
    good-count++;
  endfunction
  function void print (string str = "good-pkt");
    $display("...%05s...", str);
    super.print();
    $display("good-count = %0d", good-count);
  endfunction
endclass

```

```

eth-gen.sv
class eth-gen;
    good-pkt gpkt;
    bad-pkt bpkt;
    ill-pkt ipkt; bit[1:0]s;
    rand pkt-type s;
    constraint a-c {
        s != 2'b11;
    }
    task run(); repeat (10) begin
        case (s)
            2'b00: begin
                gpkt = new();
                assert (gpkt.randomize());
                gpkt.print();
                gen2bfm.put(gpkt);
            end
            2'b01: begin
                bpkt = new();
                assert (bpkt.randomize());
                bpkt.print();
                gen2bfm.put(bpkt);
            end
            2'b10: begin
                ipkt = new();
                assert (ipkt.randomize());
                ipkt.print();
                gen2bfm.put(ipkt);
            end
            default: $display("there is no packet");
        endcase
    endtask
endclass

```

eth-bfm.sv

```

class eth-bfm;
    eth-pkt pkt;
    task run();

```

```

eth-gen.sv
class eth-gen;
    good-pkt gpkt;
    bad-pkt bpkt;
    ill-pkt ipkt; bit[1:0]s;
    task run();
        repeat (10) begin
            s = $random; range(0:1);
            case (s)
                2'b00: begin
                    gpkt = new();
                    assert (gpkt.randomize());
                    gpkt.print();
                    gen2bfm.put(gpkt);
                end
                2'b01: begin
                    bpkt = new();
                    assert (bpkt.randomize());
                    bpkt.print();
                    gen2bfm.put(bpkt);
                end
                2'b10: begin
                    ipkt = new();
                    assert (ipkt.randomize());
                    ipkt.print();
                    gen2bfm.put(ipkt);
                end
                default: $display("there is no packet");
            endcase
        end
    endtask
endclass

```

```

forever begin
    pkt = new();
    gen2bfm.get(pkt);
    pkt.print("bfm-prints");
end endtask endclass

```

```

eth-agent.sv
class eth-agent;
    eth-gen gen;
    eth-bfm bfm;
    task run();
        forever begin
            gen = new();
            bfm = new();
        fork
            gen.run();
            bfm.run();
        join
    end
endtask
endclass

```

Top.sv

```

mailbox gen2bfm = new();
#include "eth-pkt.sv"
#include "good-pkt.sv"
#include "bad-pkt.sv"
#include "ill-pkt.sv"
#include "eth-gen1.sv"
#include "eth-bfm.sv"
#include "eth-agent.sv"

```

Module tb;

eth-agent agent;

initial begin

agent = new();

agent.run();

end

endmodule

Run.do

vlog top.sv

vsim -novopt -suppress 12140 tb

run -all

poly.sv

```

class parent;
    int a;
    virtual function void print();
        $display("a = %0d", a);
    endfunction
endclass

class child extends parent;
    int b;
    function void print();
        super.print();
        $display("b = %0d", b);
    endfunction
endclass

```

Module tb;

parent P;

child ch;

initial begin

ch = new();

ch.a = 100;

ch.b = 200;

\$cast(P, ch);

P.print();

end

endmodule

a = 100

b = 200

Enum: Enumeration

it will assign values to the english letters

typedef enum <data-type> <size> {values} <enum-variable>

Ex: typedef enum bit [1:0] {good = 2'b00, bad = 2'b01, ill = 2'b10} pkt = type;

polymorphism: poly = many, morphism = changing forms

1) object is need to be changed during run time

• If we want to achieve polymorphism means we need to declare parent methods as virtual.

• In polymorphism need to use "virtual" keyword.

• Through parent we are getting the child properties

28/02/25.

Without-polymorphism

```
eth-pkt.sv
print
constraint
good-pkt.sv
class good-pkt --
function void
constraint p-c {
    p.t = GOOD;
}
endclass
bad-pkt.sv
class bad-pkt --
function void
constraint p-c {
    p.t = BAD;
}
endclass
ill-pkt.sv
class ill-pkt extends --
function void
constraint p-c {
    p.t = ILL;
}
endclass
```

```
eth-gen1.sv
eth-bfm.sv
class eth-bfm
    eth-pkt pkt;
    good-pkt gpkt;
    bad-pkt bpkt;
    ill-pkt ipkt;
    task run();
    forever begin
        pkt = new();
        gen2bfm.get(pkt);
        if (pkt.p.t == GOOD) begin
            $cast(gpkt, pkt);
            pkt.print("bfm. good-prints");
        end
        else if (pkt.p.t == BAD) begin
            $cast(bpkt, pkt);
            pkt.print("bfm. bad-prints");
        end
        else begin
            $cast(ipkt, pkt);
            pkt.print("bfm. ill-prints");
        end
    end
endtask
endclass

eth-agent.sv
TOP.sv
mail box gen2bfm=new;
typedef enum bit [1:0]
{GOOD=2'b00, BAD=2'b01,
ILL=2'b10} pkt-type;
include -> pkt
good
bad
ill
eth-gen1
bfm
agent.sv
module tb;
    eth-agent
    agent;
    initial begin
        agent=new;
        agent.run();
    end
endmodule
```

enum-sv

```
typedef enum int {A=1, B=2, C=3, D=4} (char);
```

```
class sample;
```

```
rand char a;
```

```
function void print();
```

```
{display("a=%s", a);
```

```
endfunction
```

```
endclass
```

```
module tb;
```

```
sample s;
```

```
initial begin
```

```
s=new c();
```

```
repeat(10) begin
```

```
s.randomize();
```

```
s.print();
```

```
end
```

```
end
```

```
endmodule
```

abstract class:

- It is a virtual class, no need to create memory for the abstract class.

- All the methods will be virtual, if access these abstract class need to consider the inheritance

- According to oop's in abstract we can create write empty methods and can use those empty methods in inherited class (child class).

- If need to write the methods in abstract class and can access those methods in inherited classes

abstract-class-with-empty-method

```
eth-pkt-sv
```

```
Virtual class eth-pkt;
```

```
rand bit [55:0] preamble
```

```
"
```

```
sof;
```

```
sa;
```

```
da;
```

```
len;
```

```
rand byte payload [8];
```

```
bit [31:0] crc;
```

```
// print
```

```
virtual function void print
```

```
(string str = "eth-pkt");
```

```
endfunction
```

constraint

② eth-good-pkt.sv

```
class good-pkt extends eth-pkt;
  static int good-count;
  function new();
    good-count++;
  endfunction
  function void print(string str="good-pkt");
    $display("... %s ...", str);
    $display("preamble = %d", preamble);
    sof      sof
    sa      sa
    da      da
    len     len
    "payload = %p", payload);
    crc = %d", crc);
    $display("good-count = %d", good-count);
  endfunction
endclass
```

③ Bad-pkt.sv

same as good all will be print

③ ill-pkt.sv

same as good

④ eth-geni.sv

⑤ eth-bfm.sv

```
class eth-bfm;
  eth-pkt pkt;
  task run();
    forever begin
      pkt = new();
      geni_bfm.get(pkt);
      pkt.print("bfm-prints");
    end
  endtask
endclass
```

⑥ eth-agent.sv

⑦ top.sv

mailbot geni_bfm = new();

include all 6



Abstract class with method

① eth-pkt.sv

same as empty method

② print

virtual function void

print(string str="eth-pkt");

\$display("... %s ...", str);

\$display("preamble = %d", preamble);

sof

sa

da

len

("payload = %p", payload);

\$display("crc = %d", crc);

endfunction

→ constraint

③ good-pkt.sv

class good-pkt extends eth-pkt;

static int good-count;

function new();

good-count++;

endfunction

function void print(string str =

"good-pkt");

\$display("... %s ...", str);

super.print();

\$display("good-count = %d", good-count);

endfunction

④ bad-pkt.sv
Same as good-pkt

⑤ ill-pkt.sv
Same as good-pkt

⑥ eth-gen1

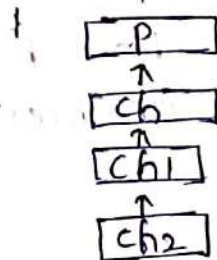
⑦ eth-blm.sv
Same as empty method

⑧ eth-agent.sv

⑨ topsv

mailbot genzblm=new1;
'include all

Multi inheritance:



multilevel inheritance:

- can create multilevel child class
- multilevel child classes will also access the parent properties, methods and constraints

Ex: class parent;

|| properties

|| methods

|| constraints

endclass

① class child extends parent;

|| these class access parent
properties, methods constraints
endclass

etc

② class child1 extends child;
|| access parent child
properties
methods
constraints
endclass

③ class child2 extends child;
|| These class access child1,
child and parent
properties
methods
constraints
endclass

Multi-level.sv

parent
class parent;
rand int a;
-function void print();
\$display("a=%0d", a);
endfunction
constraint a < 100;
a == 100;
endclass

child
parent
class child extends parent;
rand int a;
-function void print();
super.print();
\$display("parent value = %0d",
super.a);
\$display("child value = %0d", a);
endfunction
constraint c < 200;
a == 200;
endclass

child1

```
class child1 extends child;  
  rand int a;  
  function void print();  
    super.print();  
  $display("child1 value=%d", super.a);  
  $display("child1 value=%d", this.a);  
endfunction  
constraint c1-c {  
  a == 300; }  
endclass
```

child2

```
class child2 extends child1;  
  rand int a;  
  function void print();  
    super.print();  
  $display("child1 value=%d", super.a);  
  $display("child2 value=%d", this.a);  
endfunction  
constraint c2-c {  
  a == 400; }  
endclass
```

module tb;

```
  child2 ch;  
  initial begin  
    ch = new();  
    ch.randomize();  
    ch.print();  
  end  
endmodule
```

② good-pkt.sv

Same as abstraction with method

```
→ Good-pkt  
constraint p-c {  
  p-t == GOOD; }  
endclass
```

forward declaration of the class
whenever we don't consider
the order of the files, it may
lead to the error (invalid type
to avoid those error we need
to consider the forward
declarations

Syntax: typedef class <class-name:

• These forward declaration will
be applicable to classes only
not to modules

• class is a data type, module
is not a data type

① eth-pkt.sv

```
eth-pkt.sv  
rand bit [55:0] preamble,  
  sof, sa, da, len, payl
```

```
rand byte payload [0];
```

```
bit [31:0] crc;
```

```
rand pkt_type p-t;
```

2 // print

```
virtual function void print
```

```
(string str = "eth-pkt");
```

```
$display("... %s ...", str);
```

```
preamble, sof, sa, da, len, payload  
crc
```

```
$display("p-t=%s", p-t);
```

```
endfunction
```

```
// constraint
```

③ Bad-pkt.sv

same as abstraction

→ Bad-pkt

```
constraint p-c {
```

```
  p-t == BAD; }
```

```
endclass
```

④ JLL-pkt.sv

same as abstraction

→ JLL-pkt

constraint p-cs

p-t == JLL;

endclass

⑤ eth-genl.sv - 25/02

eth-genl.sv

constraint a-cs

"s1 = 2'b";

"}

endclass

⑥ eth-bfm.sv

class eth_bfm;

eth_pkt pkt;

good_pkt gpkt;

bad_pkt bpkt;

ill_pkt ipkt;

task run();

forever begin

pkt = new();

gen_bfm.get(pkt);

if(pkt.p-t == GOOD) begin

\$cast(gpkt, pkt);

pkt.print("bfm gpkt prints");

end

else if(pkt.p-t == BAD) begin

\$cast(bpkt, pkt);

pkt.print("bfm-bad prints");

end

else begin

\$cast(ipkt, pkt);

pkt.print("bfm-ill prints");

end

end

endtask

endclass

⑦ eth-agent.sv

class eth-agent;

eth_gen gen;

eth_bfm bfm;

task run();

gen = new();

bfm = new();

task

gen.run();

bfm.run();

endtask

endclass

⑧ eth-top.sv

mailbox gen2bfm = new();

typedef enum bit[1:0] {GOOD = 1'b01,

BAD = 1'b00, JLL = 1'b10} pkt_t;

typedef class eth_pkt;

" good_pkt;

" bad_pkt;

" ill_pkt;

" eth_gen;

" eth_bfm;

" eth_agent;

'include "eth-genl.sv"

" eth_bfm

" eth_pkt

" good_pkt

" bad_pkt

" ill_pkt

" eth_agent

module tb;

eth_agent agent;

initial begin

agent = new();

agent.run();

end

endmodule

eth-common.sv

```
class eth-common;  
    static rand bit-type s;  
endclass
```

// parameterization

In verilog $\rightarrow$ parameter <NAME> = <value>;

by using # we can override the parameters

in verilog we have only value parameterization

In system verilog $\rightarrow$ parameter int <NAME> = <value>;

* In system verilog we have value parameterization and type parameterization.

1. value parameterization

Ex: class sample #(parameter int WIDTH=5);

```
    rand bit [WIDTH-1:0] a;  
    function void print();
```

```
        $display("a=%b", a);
```

```
    endfunction
```

```
endclass
```

```
module tb;
```

```
    // parameter int WIDTH=5;
```

```
    parameter int WIDTH=8;
```

```
    sample #(WIDTH) s;
```

```
    initial begin
```

```
        s = new();
```

```
        repeat (5) begin
```

```
            s.randomize();
```

```
            s.print();
```

```
        end
```

```
    end
```

```
endmodule
```

* In system verilog we can pass the value as a parameter and if we want to override the value means we need to consider #

2. Type parameterization

* In system verilog type parameterization is also available.

Ex: class sample #(parameter
 type T=int);

```
    rand T a;
```

```
    function void print();
```

```
        $display("a=%d", a);
```

```
        $display("a=%b", a);
```

```
    endfunction
```

```
endclass
```

```
module tb;
```

```
    // sample #(int) s;
```

```
    sample #(byte) s;
```

```
    initial begin
```

```
        s = new();
```

```
        repeat (5) begin
```

```
            s.randomize();
```

```
            s.print();
```

```
        end
```

```
    end
```

```
endmodule
```

∴ Type T = <data-type>

Stack:

LIFO → last in - first out

int width=10, real, width=15, byte, width=20

Stack:sv

class stack #(type T=int, int width=10);

`que[s];

T value;

function void push();

`for(int i=0; i<width; i++) begin

value = \$random-range(10,50);

que.push-back(value);

end

\$display("stack=%p", que);

endfunction

function void pop();

`for(int i=0; i<width; i++) begin

value = que.pop-back();

\$display("stack=%p", que);

end

endfunction

function void print();

endfunction

endclass

module tb;

Parameter int width=10;

`stack #(int,width) s;

`ifdef INT

initial begin

\$display("handling with int data");

end

stack #(int,10) s;

`elseif REAL

initial begin

\$display("handling with real data");

end

stack #(real,15) s;

`else

initial begin

\$display("handling with
byte data");

end

stack #(byte,20) s;

`endif

initial begin

s = new();

s.push();

s.pop();

end

endmodule

Q: Implement FIFO and LIFO basing on control signal

* Types of parameterization class.

1. Non parameterization in Parent and Non parameterization in child.
2. Parameterized parent with non parameterized child.
3. Non parameterized parent with parameterized child.
4. Parameterized parent with parameterized child.

1. Non parameterization in parent
and Non parameterization in child
Non → parent, child

Syntax: class parent;
properties, methods, constraints
endclass
class child extends parent;
properties, methods, constraints
endclass

Ex: class parent;
int a;
function void print();
\$display("a=%0d", a);
endfunction
endclass

class child extends parent;
int b;

function void print();
super.print();
\$display("b=%0d", b);
endfunction
endclass

module tb;

child ch;

initial begin

ch = new();

ch.a = 100;

ch.b = 200;

ch.print();

end

endmodule

2. parameterization in parent
And Non-parameterization in child
[Non-child, parent]

Syntax: class parent #(type T = int, int width = 8)
"pro, meth, con"; endclass

class child extends parent #(int, 5);
"pro, meth, con"; endclass

Ex: class parent #(type T = int,
int width = 8);

T[width-1:0] a;

function void print();
\$display("a=%0d", a);
\$display("a=%0b", a);
endfunction
endclass

class child extends parent #(bit, 8)
int b;

function void print();
super.print();
\$display("b=%0d", b);
endfunction
endclass

module tb;

child ch;

initial begin

ch = new();

ch.a = 100;

ch.b = 200;

ch.print();

end

endmodule

3. Non parameterization parent
and parameterization child
[child, Non-parent]

syntax: class parent;
"pro, met, con; endclass
class child # (type T=int,
int width=5) extends parent;
"pro, meth, const; endclass

Ex: class parent;
int a;
function void print();
\$display("a=%0d", a);
endfunction
endclass
class child # (type T=int,
int width=5) extends parent;
T [width-1:0] b;
function void print();
super.print();
\$display("b=%0d", b);
\$display("b=%b", b);
endfunction
endclass

module tb;
child # (bit, 8) ch;
initial begin
ch = new();
ch.a = 100;
ch.b = 200;
ch.print();
end
endmodule

4. parameterization parent
parameterization child
[child, parent]

syntax: class parent # (type T=int, int
width=5);
"pro, meth, const; endclass
class child # (type T=C → No
parameterization parent or

syntax: class parent # (type T=int,
int width=5);
T [width-1:0] a;
function void print();
\$display("a=%0d", a);
\$display("a=%b", a);
endfunction
endclass

" class child # (type T=C=int,
width=C=5) extends
parent # (T=C, width=C);
class child # (type T=C=int,
int width=C=5) extends parent
(bit, 5);

T=C [width=C-1:0] b;
function void print();
super.print();
\$display("b=%0d", b);
\$display("b=%b", b);
endfunction
endclass

module tb;
child # (bit, 8) ch;
ch = new();
ch.a = 31;
ch.b = 200;
ch.print; end endmodule

Interface class

- Need to mention the interface keyword before the class.
- Interface class have only pure virtual method
- Interface class doesn't allow the inheritance
- We can't create the child class by using extend keyword.
- Here if i want to create class means we need to use this implements keyword
- If we create child class using abstract class, it is not necessary to use all the virtual methods.
- In inheritance child class whatever the methods we mentioned in the interface we need to use that methods
- Interface class only possible methods only, properties constraints not possible.
- In abstract class we can pass empty method and normal methods
- In interface class need to pass only empty pure virtual methods

Syntax: interface class parent;

pure virtual function void print();

endclass

class child implements parent;

virtual function void print();

~functionality

end-function

end class

Interface .sv

Interface class parent;

pure virtual function void print();

pure virtual function void print();

endclass

```

class child implements parent();
virtual function void print();
display("printing the dots from print");
endfunction
virtual function void print();
display("printing the dots from print");
endfunction
endclass

```

```

module tb;
child ch;
initial begin
    ch = new C;
    ch.print();
    ch.print();
end
endmodule

```

Nested class
class inside the class

Ex: class parent;

child class;

class child1;

class child2;

static int a;

static function void print();

display("a = %d", a);

endfunction

endclass

endclass

endclass

endclass

module tb;

initial begin

parent::child::child1::child2::a = 100;

parent::child::child1::child2::print();

end

endmodule

∴ for accessing the nested class
variable or function need to
consider the scope resolution
operator

constant

constant class properties:

- In class don't want to change those properties means indeed to declare those properties as const class properties
- If i declare class properties as constant then those are read only properties
- we need to use "const" keyword for making the properties as const

Types of constant properties

1. Global

2. Instance constant property

Global constant property:

- The constant property value will be assigned in the class, that value we can't change.

Instance constant property:

- The constant property value can be modified by using the methods.

Program: constant.sv

```
class sample;
```

```
const int count = 20; // Global const property
```

```
const int count1; // instance const property
```

```
function new (int count1);
```

```
    this.count1 = count1;
```

```
    // this.count = count1;
```

```
endfunction
```

```
function void print();
```

```
    $display("Global count = %0d", count);
```

```
    $display("instance count = %0d", count1);
```

```
endfunction
```

```
endclass
```

```
module tb;
```

```
    sample s;
```

```
    initial begin
```

```
        s = new(100);
```

```
        s.print();
```

```
        // s.count = 200;
```

```
        // s.count1 = 300;
```

Variable scope:

1. Global scope
2. class or module scope
3. Task or function scope

Global scope means:

If we declare the properties or methods outside the class or module, those are known as global scope properties or methods.

- By using `$unit and (::)` scope resolution operator we can access the global scope variables.

② class scope:

If we declare properties or methods inside the class, those are having class scope.

③ ^{Task} module scope: Declare any property inside the task, we can use that property inside the task anywhere.

④ ~~task~~ module scope: Declare any methods or properties inside the module are having module scope.

⑤ function scope: Declare any properties inside the function, those properties we can use anywhere inside the function.

Scope.sv.

```
int count;
class sample;
    int count;
    int a;
function set ();
    a = this.count;
endfunction
endclass

module tb;
    int count;
    sample s;
    initial begin
        s = new X;
```

```
$Unit :: count = 10;
```

```
s.count = 20;
```

```
s.set ( );
```

```
count = 30;
```

```
$root.tb.print.count = 40;
```

```
print(40);
```

```
$display("Global-count = %d", $Unit :: count
```

```
$display(" class-count = %d", s.count);
```

```
$display(" class-count = %d", s.a);
```

```
$display(" module-count = %d", count);
```

```
$display(" function-count = %d", $root.tb
    print.count);
```

```
$display("function. count = %d", $root.b. print.b);
```

```
end
```

```
function void print (int count);
```

```
int b;
```

```
b = count;
```

```
end function
```

```
endmodule
```

object copy:

- we are copying one object to the other object

- we have 4 ways to copy the objects

1. copy by handle

2. shallow copy

3. deep copy

4. \$cast

① copy by handle:

- copy by handle we are copying the memory of one object to the other object

```
handle.sv
```

```
class sample;
```

```
int a;
```

```
int b;
```

```
function void print (string str);
```

```
$display("prints the data from object = %s", str);
```

```
$display("a = %d", a);
```

```
$display("b = %d", b);
```

```
end function
```

```
endclass
```

```
module tb;
```

```
sample s1, s2;
```

```
initial begin
```

```
s1 = new();
```

```
handle.sv
```

```
s1.a = 100;
```

```
s2.b = 200;
```

```
s1.print("s1");
```

```
s2 = s1;
```

```
s2.print("s2");
```

```
s1.a = 10;
```

```
s1.b = 20;
```

```
s1.print("s1");
```

```
s2 = s1;
```

```
s2.print("s2");
```

```
s1.a = 10;
```

```
s1.b = 20;
```

```
s1.print("after modifying s1");
```

```
s2.print("after modifying s1-s2");
```

```
s2.a = 300;
```

```
s1.b = 400;
```

```
s1.print("after modifying s2-s1");
```

```
s2.print("after modifying s2");
```

```
end
```

```
endmodule
```

* syntax `s2 = s1;`

In copy by handle both the objects are using the same memory, so if modify any object it will be reflected on the other object.

2) Shallow copy: In shallow copy consider 2 different memory for 2 obj and i will perform the copy.

- shallow is applicable for only non-object properties
- for object properties shallow copy will do copy by handle

Shallow copy:

```
class parent;
    int count;
endclass

class sample;
    parent p=new();
    int a;
    int b;
    function set(int y);
        p.count = y;
    endfunction

    function void print(string str);
        $display("printing the data from
            object = %s", str);
        $display("count = %d", p.count);
        $display("a = %d", a);
        $display("b = %d", b);
    endfunction
endclass

module tb;
    sample s1, s2;
    initial begin
        s1 = new();
        s2 = new();
        s1.a = 300;
        s2.b = 400;
        s2.set(100);
        s1.print("after modifying s2 -> s1");
        s2.print("after modifying s2");
    // changing the s1 value
        s1.a = 10;
        s2.b = 20;
        s1.set(300);
```

```
s1.print("after modifying s1");
s2.print("after modifying s1 -> s2");
end
endmodule
```

③ * Deep copy

we need to create a copy method basing on that copy method we need to perform the

deep sv:

```
class parent;
    int count;
endclass

class sample;
    parent p=new();
    int a; int b;
    function set(int y);
        p.count = y;
    endfunction

    function void print(string str);
        $display("printing the data from
            object = %s", str);
        $display("count = %d", p.count);
        $display("a = %d", a);
        $display("b = %d", b);
    endfunction

    function void copy(output samp
        temp);
        temp = new();
        temp.a = this.a;
        temp.b = this.b;
        temp.p.count = this.p.count;
    endfunction
endclass
```

```

Module -16;
Sample s1, s2;
initial begin
    s1 = new C1;
    s2 = new C1;
    s1.a = 100;
    s1.b = 200;
    s1.set(10);
    // s2 = new s1;
    s1.copy(s2);
    s1.print("s1");
    s2.print("s2");

```

sample class
object properties
non-object properties

copy by handle
if change values in any object it will be reflected on the other object

s1

s2

shallow copy:

class sample
object properties
non-object properties

for object properties
if change any value with one object it will be replaced on the other object.

• Non-object properties. If change value with one object it won't be reflected on the other object.

s1

s2

// Modifying values from s2

s2.a = 300;

s2.b = 400;

s2.set(100);

s1.print("after modifying s2 -> s1");

s2.print("after modifying s2");

// changing the s1 values

s1.a = 10;

s2.b = 20;

s1.set(300);

s1.print("after modifying s1");

s2.print("after modifying s1 -> s2");

end

endmodule

04/03/25 - 20

④ \$ cast: we have 2 types of casting

1. Static or type casting

2. dynamic casting

* Static - casting or type casting

• we can convert the data from one data type to another data type

syntax: <data-value> = <converted-data-type>(value);

* consider a byte variable a and convert that byte value into a string

→ byte a;

string str;

str = string(a);

Dynamic casting: In dynamic casting need to copy the one object contents into the other.

for dynamic casting both the object having the relation of inheritance.

Ex: \$cast (destination-object, source-object);

dyn-cast:sv

class parent;

int a;

virtual function void print();

{display("a=100", a);

endfunction

endclass

class child extends parent;

int b;

function void print();

{super.print();

{display("b=100", b);

endfunction

endclass

class child1 extends parent;

int c;

function void print();

{super.print();

{display("c=100", c);

endfunction

endclass

module tb;

child ch;

parent p;

child1 ch1;

initial begin

ch = new();

p = new();

ch1 = new();

ch.a = 100;

ch.b = 200;

if (\$cast(p, ch)) {display("casting done");

if (\$cast(ch, p)) {display("casting done"); // casting is not possible

* casting is only possible for
child to parent

• casting is not possible for
parent to child

• casting is not possible for
child to child

Ex: class parent;

// prop, methods, constraints

endclass

class child extends parent;

// prop, meth, constraints

endclass

class child1 extends parent;

// prop, method, constraints;

endclass

* // \$cast(p, ch); // casting possible

\$cast(p, ch); // casting possible

\$cast(ch, p);
\$cast(ch1, p);
\$cast(ch, ch1);
\$cast(ch1, ch);

} casting not
possible

\$cast(p, ch)
↓ ↓
source destination

```

else display('casting not done');
p.print();
// if ($cast(ch1, ch)) $display('casting done');
if ($cast(ch1, ch)) $display('casting done'); // casting is not possible
else $display('casting not done');
// ch1.print();
ch1.print();
end
endmodule

```

Fork Join

- fork will start all the process running concurrently
- join will ends all the processes

Statement: Any line ends with semi colon
Colon is known as statement

Process: Any thing inside the begin and end will consider as process & if there is no begin and end means each statement will consider as a process.

- * In verilog a system verilog the execution style of the code is sequentially
- * Achieve concurrent nature need to consider fork join.

Fork join: fork will start all the process running concurrently
join will ends all the process

Fork join family: In fork join family have - 3

1. fork join
2. fork join-any
3. fork join-none

programm.

Initial begin

#10; // 1 process

\$display("entry 1"); // 2

#5 \$display("entry 2"); // 3

begin // 4

\$display("entry 3");

#2;

#5 \$display("entry-4");

end

#2; \$display("entry-5");

end

→ 4 processes.

Fork-join: It will process concurrently.

After completing all the statements inside the fork join it will execute the outside statements.

Ex: module tb;

initial begin

\$display("%t: entry-1", \$time); // 0ns

#10 \$display("%t: entry-2", \$time); // 10ns

fork

\$display("%t: entry-3", \$time); // 10ns

#10 \$display("%t: entry-4", \$time); // 20ns

#2; \$display("%t: entry-5", \$time); // 10ns

begin

\$display("%t: entry-6", \$time); // 10ns

#3; \$display("%t: entry-7", \$time); // 13ns

\$display("%t: entry-8", \$time); // 13ns

end

begin

\$display("%t: entry-9", \$time); // 10ns

end

join

#20;

\$display("%t: entry-10", \$time);

end

endmodule

Fork-join_any: If any one process is completed, it will start executing the outside statements.

Module tb;

initial begin

\$display("%t: entry-1", \$time); 0

#10 \$display("%t: entry-2", \$time); 10ns

fork

\$display("%t: entry-3", \$time); 10ns

#10 \$display("%t: entry-4", \$time); 20ns

#5; \$display("%t: entry-5", \$time); 10ns

begin

\$display("%t: entry-6", \$time); 10ns

#3; \$display("%t: entry-7", \$time); 13ns

\$display("%t: entry-8", \$time); 13ns

end

begin

\$display("%t: entry-9", \$time);

10ns

end

join_any

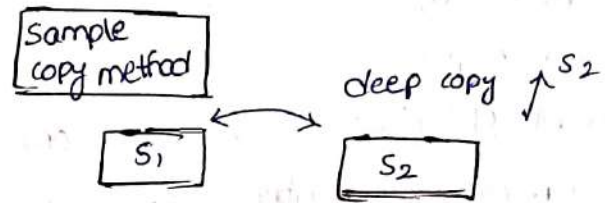
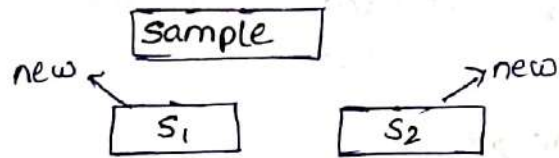
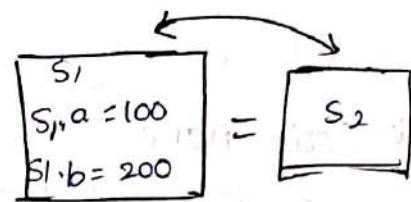
#5;

```

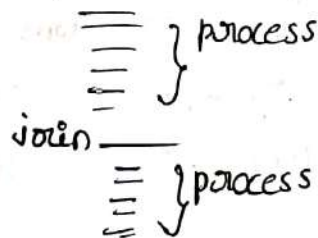
$display("10: entry-10", $time); 30ns
#1;
$display("11: entry-11", $time); 31ns
#5 $display("12: entry-12", $time); 36ns
$display("13: entry-13", $time); 36ns
end
endmodule

```

class sample

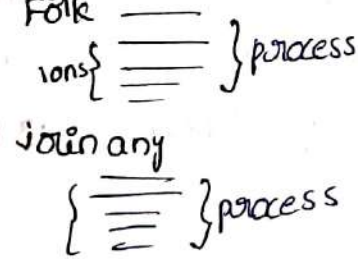


Fork



It will complete all the processes than only it will state the outside process

Fork



If any 1 process is completed, it will start existing the outside statement

```

Any.sv
Module tb;
reg clk, valid, ready;
always #5 clk = ~clk;
initial begin
    clk = 0;
    valid = 0;
    ready = 0;
    #20 valid = 1;
    #30 ready = 1;
end
initial begin
    forever begin
        fork
            begin
                @(posedge clk);
                wait (ready == 1);
                $display("ready is happened %0t", $time);
            end
        begin
            repeat(51 @(posedge clk);
            if (ready == 0)
                $display("ready is not happened %0t", $time);
            end
        join-any
    end
    initial begin
        #200; $finish; end
endmodule

```

→ valid → ready

10ns → valid

100ns → ready is happened or not

read → during

10 clk cycles need to check ready is happen or not.

* 2 process

{ waiting
 Informing } ←

fork-join-any

fork-join-None;

fork-join-None will execute the outside statements along with the inside statements.

None-sv

module tb;

initial begin

\$display("%ot: entry-1", \$time); 0ns

#10 \$display("%ot: entry-2", \$time); 10ns

fork

\$display("%ot: entry-3", \$time); 10ns

#10 \$display("%ot: entry-4", \$time); 20ns

#20 \$display("%ot: entry-5", \$time); 10ns
begin

\$display("%ot: entry-6", \$time); 10ns

#30 \$display("%ot: entry-7", \$time); 13ns

\$display("%ot: entry-8", \$time); 13ns

end

begin

\$display("%ot: entry-9", \$time); 10ns

end

join-~~none~~

#5;

\$display("%ot: entry-10", \$time); 15ns

#1;

\$display("%ot: entry-11", \$time); 16ns

#5 \$display("%ot: entry-12", \$time); 21ns

\$display("%ot: entry-13", \$time); 21ns

end

endmodule

o/p: 1 2 3 5 6 9 7 8 10 11 4 12 13

Ex: Bank → applying ATM card
→ Application → 30 min
30 min → it is ok

2-pro fork

10ns { } process }
 }
 join_name

10ns { } process }

Fork-join-None.sv: It will execute all the processes concurrently along with the inside process fork join none will start executing the outside process also

```

// TASK run
fork
  drive_tx(C);
join-None
endtask

```

None.sv

```

module tb;
  int i, j, value;
  initial begin
    forever begin
      fork

```

```

    $display("process = %0d is started at %0t",
              i, i++, $time);
    $display("process = %0d is complete at %0t",
              j++, $time);

```

```

    join-None
    value = $random-range(1, 15);
    $display("value = %0d, %0t", value, $time);
    #value;
    end
    end
    initial begin
      #50;
      $finish(1);
    end
  endmodule

```

Op: value = 11, 0
process = 0 started at 0
0 complete 5

Value = 3, 11

Proces = 1 start 11

Value = 11, 14

pro = 2 is start 14

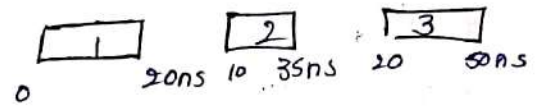
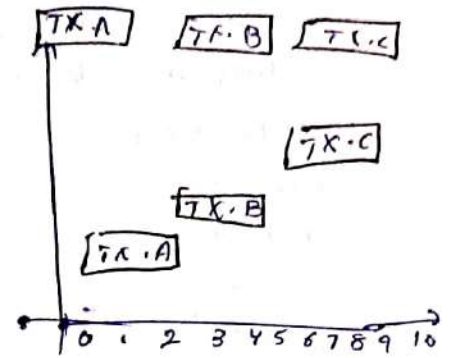
pro = 1 comp 16

pro = 2 is com 19

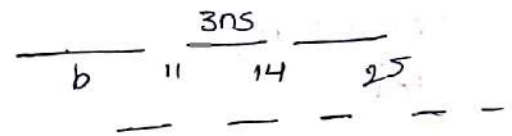
Value = 8, 25

pro = 3 is st 25

pro = 3 comp 30



1 → 0n → P1
→ 10ns P2
20ns P1 and → 20ns P3
→ 35ns



~~Fork~~

P1;

P2;

P3;

} kill the processes

join-None

disable fork;

~~Fork~~

P1;

P2;

P3;

} execute all the process

join

disable fork

Any.sv

```
module tb;
```

```
reg clk, req, ack;
```

```
always #5 clk = ~clk;
```

```
initial begin
```

```
clk = 0;
```

```
req = 0;
```

```
ack = 0;
```

```
#10;
```

```
ack = 1;
```

```
#50;
```

```
req = 1;
```

```
#200;
```

```
$finish;
```

```
end
```

```
initial begin
```

```
forever begin
```

```
fork
```

```
begin
```

```
@(posedge clk);
```

```
wait (req == 1);
```

```
$display("req is happened  
%t", $time);
```

```
end
```

```
begin
```

```
repeat(3) @(posedge clk);
```

```
$display("req is not happened  
%t", $time);
```

```
end
```

```
join-any
```

```
disable fork;
```

```
end
```

```
end
```

```
// initial begin
```

```
// #200;
```

```
// $finish;
```

```
// end
```

```
endmodule
```

Disable fork:

- It will disable the fork
- This disable fork need to consider in fork join-any,
- Some cases can consider this disable fork join-none it will kill these processes.

Nested fork

```
fork : HF
```

```
fork : MF
```

```
fork : LF
```

```
join
```

```
join
```

```
join
```

```
disable LF;
```

```
disable HF
```

```
disable fork
```

Task and functions:

- It is a piece of code.
- Readability and reusability will happen with task and function
- Pass both inputs and output arguments.

```
// function <return-type> function_name  
(<input arguments>, <output arguments>)
```

```
// functionality
```

```
endfunction
```

Types of functions:

with arguments with return type
with argument without return type
without arguments with return type
without arguments without return type

|| Task:

syntax: task <task_name>(input and op argum)
|| Task functionality
endtask

Q: write a function for copying the contents of apb_tx class with return type

• pwritu	prdata
paddr	pse1
pwdata	perr

* function apb_tx copy();

```
    apb_tx temp;  
    temp = new();  
    temp.pwritu = pwritu;  
    temp.paddr = paddr;  
    temp.pwdata = pwdata;  
    temp.prdata = prdata;  
    temp.pse1 = pse1;  
    temp.perr = perr;  
    return temp;  
endfunction
```

|| with arguments and with return type.

function apb_tx copy(input apb_tx at);

```
    apb_tx temp;  
    temp = new();  
    temp.pwritu = at.pwritu;  
    temp.paddr = at.paddr;  
    temp.pwdata = at.pwdata;  
    temp.prdata = at.prdata;  
    temp.pse1 = at.pse1;  
    temp.perr = at.perr;  
    return temp; endfunction
```

```
function void copy(input apb_tx ap,  
    apb_tx temp = new();  
    temp.pwritu = ap.pwritu;  
    temp.paddr = ap.paddr;  
    temp.pwdata = ap.pwdata;  
    temp.prdata = ap.prdata;  
    temp.pse1 = ap.pse1;  
    temp.perr = ap.perr;  
endfunction;
```

apb2: copy(apb1);

Passing the arguments

1. Pass by value
2. Pass by reference

① Pass by value:

~~pass the argument~~

- we are passing the value through the arguments
- whatever arguments is there we need to create memory for that arguments

pass-by-value.sv

Module tb;

int a,b,c;

function void sum(input int a,b,
output int c);

a=a+10;

c=a+b;

endfunction

initial begin

a=10;

b=20;

\$display("a=%0d, b=%0d, sum=%0d",
a,b,c);

sum(a,b,c);

\$display("a=%0d, b=%0d, sum=%0d",
a,b,c);

end

endmodule

O/p: a=10, b=20, sum=0
a=10, b=20, sum=40

② Pass by reference

- we didn't create memory for the values, we will refer the memory of the existed values:

- If consider pass by reference there is chance data modification

② pass-by-ref.sv

Module tb;

int a,b,c;

function automatic
int sum(ref int a,b);

a=a+10;

return a+b;

endfunction

initial begin

a=10;

b=20;

\$display("a=%0d, b=%0d",
a,b,c);

sum=a+b;

c=sum(a,b);

\$display("a=%0d, b=%0d",
a,b,c);

sum=a+b;

end

endmodule

O/p: a=10, b=20, sum=0

a=20, b=20, sum=40

constant refer

If we mention those properties as constant means we can't modify those values.

const-pass-by-ref.sv

Module tb;

int a,b,c;

function automatic int sum
(const ref int a,b);

// a=a+10;

return a+b;

endfunction

initial begin

a=10; b=20;

\$display("a=%0d, b=%0d, sum=%0d",
a,b,c);

```

c = sum(a, b);
$display("a = %d, b = %d, sum = %d", a, b, c);
end
endmodule

```

O/p: a = 10, b = 20, sum = 0
a = 10, b = 20, sum = 30

06/03/25 - 22

System task and function

For printing:

\$display: display the contents at once.
code execution line reaches to statement, immediately it will display.

\$write: same as \$display
it will print all the statements in single line

\$strobe: It will display the contents at the end of the time step.

\$monitor: Any change happened in the list mention in the monitor it will display at the end of the time step.

$\$display = \$write + \backslash n$

Randomization:

\$random: Generate 32 bit signed value

\$urandom: Generate 32 bit unsigned value

\$urandom-range: Generate 32 bit value b/w the range

For file handling:

\$fopen: open the file

\$fclose: close the file

\$fdisplay: write the data into the file

\$fmonitor: same as \$monitor but the data will be displayed as write into the file

\$fgets: used to read the data from the file

\$time: provide the current simulation time

\$realtime: current simulation time in real format

For memory:

\$readmemb: write the data into memory from the hexa-file

\$writememb: Read the data from the memory and it will create an o/p

\$readmemb: write the data into the memory from the ^{hexa-file} binary-file

\$writememb: Read the data from the memory and it will create an o/p ^{hexa-file} binary-file

User provided value: \$value \$plusargs

bit vectors: \$countone \$onehot \$onehot0

Random strings generation: \$sf3format \$sf3formatf

User define data types: 1. class 3. enum 5. Union
2. typedef 4. struct

1. Class: class is a user defined datatype, class consists of properties, methods and constraints

2. typedef: user will create the data by using existed data types.

Ex: typedef bit [3:0] nibble;

3. Enum: It will assign value to the english letters

typedef enum <data-type> { letter = value1, letter = value2, ... }
enum-type

Ex: typedef enum int { a=0, b, c, d, e, f, g } alpha;

Ex: typedef enum int { a=0, b, c, d=20, e, f, g };

4. struct: collection of heterogeneous data

syntax: struct {
 data-type1 data1;
 data-type2 data2;
 data-type3 data3;
} struct-type;

5. Union: collection of heterogeneous data

It is same as struct. In struct create memory for all the properties. In union it will create only memory for highest data variable memory, this memory will be used by the every element present in the union.

IPS: Inter process synchronizations

Event

Mailbox

Semaphore

Events: used to synchronize the processes

In events we have 2 main parts

In Verilog: 1. Triggering (->) 2. Waiting (@)

system waiting events.

1. triggering (wait keyword);
2. waiting (wait keyword);

- Before the triggering if we mention wait that wait will be cleared.
- If mention the wait after triggering, if the wait is in same time step than that wait will be cleared
- If it is another time step means that wait won't be cleared

Struct - SV

typedef struct {

int a;
real b;
bit[3:0] c;
} struct - n;

module tb;

struct-n s;
initial begin

s.a = 10;
s.b = 10.05;
s.c = 15;

\$display("s=%p", s);

end
endmodule

o/p: s = {a: 10, b: 10.05, c: 15}

Event1 - SV

module tb;

event e1, e2;

initial begin

#0.1;

@(e1);

\$display("entry-1");

→ e2;

end

initial begin

→ e1;

\$display("entry-2");

end

initial begin

@(e2);

\$display("entry-3");

end

endmodule

Union - SV

typedef union {

int a;

real b;

bit [# s = {a: -171798705,

b: 10.05, c: 15}, 3:0] c;

} union - n;

module tb;

union-n s;

initial begin

s.a = 10;

s.b = 10.05;

s.c = 15;

\$display("s=%p", s);

end

endmodule

o/p: s = {a: -171798705, b: 10.05, c: 15}

Event2 - SV

module tb;

event e1, e2;

initial begin

// @(e1);

#0.4;

wait(e1.triggered);

\$display("entry-1");

→ e2;

end

initial begin

→ e1;

\$display("entry-2");

end

initial begin

// @(e2);

wait(e2.triggered);

```

display("entry-3");
end
endmodule

```

o/p: entry-2 entry-2
 entry-1
 entry-3

-Attendance:

```

typedef struct {
    string name;
    int roll-number;
} attendance;

```

```

module tb;
    attendance a[10];
    int value;
    initial begin
        foreach (a[i]) begin
            value = $urandom_range(65, 90);
            a[i].name = $format("%s", value);
            a[i].roll-number = $urandom_range(1, 101);
        end
    end
    display("attendance = %p", a);
end
endmodule

```

o/p: attendance = { { name: "y", roll-num: 17,

semaphore:

- 23

- semaphore is an inter processes synchronizer
 - It is used to synchronize the process basing on the availability of the source.
 - whenever 2 or more processes using the same resource semaphore will provide the synchronization
- In semaphore have keys.

syntax: semaphore semaphore-name = new({keys});

Methods in semaphore:

1. New: used to create the size or NO. of resource available

syntax: semaphore semaphore-name = new({keys});

2. Num: NO. of resources available

syntax: semaphore-name.num();

3. put: put the keys into semaphore;

syntax: semaphore-name.put({key});

Blocking nature

4. Get: Get the keys from the semaphore

syntax: semaphore-name.get({key});

Blocking nature

5. try-put: put the keys into the semaphore

syntax: semaphore-name.try-put({key});

Non-blocking nature

6. Try-get: Get the keys from the semaphore

syntax: semaphore-name.get({key});

Non-blocking nature

7. Peek: tells about the first key.

semaphore-name.key();

-Application:

- whenever having multiple agents, if all the agents wanted to communicate with out through interface at same time there is a chance of data loss will be happened
- to avoid these thing we will consider semaphore to make synchronization b/w the agents.

Semaphore.v

semaphore smp=new();

module tb;

initial begin

smp.get(1);

\$display("%ot: process1 is started", \$time);

writec();

\$display("%ot: process1 is completed", \$time);

smp.put(1);

end

initial begin

#2;

\$display("%ot: process2 is waiting for key", \$time);

smp.get(1);

\$display("%ot: process2 is completed", \$time);

smp.put(1);

end

task writec();

\$display("writes are going on");

#20;

endtask

endmodule

0/P: 0: process1 started
writes going on

2: pr2 is waiting for key

20: pr1 is completed

20: pr2 is started
writes going on

Extern keyword: whenever we use the methods outside the class or module we can use this "Extern" keyword to access those methods inside the class

extern;sv

class eth-pkt;

rand bit [55:0] preamble

src, sa, da, len, payload, crc;

Extern function void print();

constraint a-c {

len inside {5:10};

payload.size == len;

}

endclass

function void eth-pkt::print();

\$display("preamble=10d", preamble);

src, sa, da, len, payload, crc);

module tb;

eth-pkt pkt;

initial begin

pkt = new();

pkt.randomize();

pkt.print();

end

endmodule

-28

Constraints: Limitations of the signals

In any file will consider the signals with some specifications, in a specification we have some limitations

- In developing those limitations we need to consider the constraints.

Ex: int a i want to generate a value b/w 100 to 200

if i want to randomize that it will generate the value

b/w -2^{31} to $2^{31}-1$

So if i want to consider this specification means i need to consider the constraints.

CONSTRAINTS

Types of constraints: 2 1. Inline 2. Inclass constraints

Inline constraints:

• The constraint which is mentioned in the same line of randomization

• In inline constraint we use "with" keyword for writing the constraint in the randomization line.

Ex: object.randomize with {constraint-logic};

Inline:sv

```
class sample;
  rand int a;
  function void print();
    $display("a=%0d", a);
  endfunction
endclass
```

```
module tb;
  sample s;
  initial begin
    s = new();
    repeat(10) begin
      s.randomize() with {a > 100; a < 200;};
      s.print();
    end
  end
endmodule
```

constraint:sv

```
class sample;
  rand int a;
  function void print();
    $display("a=%0d", a);
  endfunction
  constraint a_c {
    a > 100;
    a < 200; } endclass
```

```
module tb;
  sample s;
  initial begin
    s = new();
    repeat(10) begin
      s.randomize(); s.print();
    end end
  endmodule
```

o/p: a = 199	a = 120
a = 164	a = 167
a = 121	a = 171
a = 179	a = 104

Inclass constraint: The constraint which are mentioned inside the class.

Ex: class class_name;
 // constraint
endclass

Types of inclass constraint:

we have 8 types of inclass constraints.

1. simple constraints
2. Distributive constraints
3. if-else
4. Implicit
5. iterative
6. soft
7. unique
8. variable order constraint

1. simple constraints: The constraints which are mentioned a simple mathematical expression is known as simple constraint.

Syntax: constraint constraint-name {

simple mathematical expressions }

Q: Consider 3 variable a, b and c write a constraint that a is bigger than b and c is bigger than b.

Simple-Solver

```
class sample;
```

```
rand int a,b,c;
```

```
function void print();
```

```
$display("a=%d, b=%d, c=%d", a,b,c);
```

```
endfunction
```

```
constraint a-c {
```

```
a > b;
```

```
b > c;
```

```
b > 100;
```

```
b < 1500;
```

```
c > 100;
```

```
c < 150;
```

```
// a inside [100:150];
```

```
// b inside [100:150];
```

```
// c inside [100:150];
```

```
a > b;
```

```
b > c; }
```

```
endclass
```

```
module tb;
```

```
sample s;
```

```
initial begin
```

```
s = newc();
```

```
repeat(10) begin
```

```
s.randomize();
```

```
s.printc();
```

```
end
```

```
end
```

```
endmodule
```

o/p: a=117, b=112, c=106

a=136, b=115, c=102

a=105, b=103, c=102

a=115, b=104, c=103

2. Distribution:

- Basing on the weightage it will generate the random values
- In distributive constraint we use "dist" keyword
- In this constraint we need to pass the weightage for values

a dist {value1::<weightage1>,
value2::<weightage2>,
value3::<weightage3>,...}

Ex: a dist {100::2, 200::5, 300::1, 400::3};

for ranges

a dist {range1::<weightage1>,
range2::<weightage2>,...}

Ex: a dist {[1:10]:1/2, [50:100]:1/5};

Ex: a dist {100::2, 200::5, 300::1, 400::3};

weightage will be 1

- weightage will be 1 --> if 1 randomize the value 1 times

→ 100 2 times

200 5 times

300 1 times

400 3 times

- If my values or ranges having the same weightage means i no need to consider the dist keyword so i consider the inside keyword

Dist.sv

```
class sample;  
    rand int a;  
    function void print();  
        $display("a=%04",a);  
    endfunction  
    constraint a-c {  
        a dist {100::2, 200::5,  
                300::1, 400::3};  
    }  
endclass
```

```
module tb;  
    sample S;  
    initial begin  
        S = new();  
        repeat(5) begin  
            S.randomize();  
            S.print();  
        end  
    end  
endmodule
```

o/p: a=400 a=200
a=200 a=100
a=200 a=200...

Dist1.sv

```
class sample;  
    rand int a;  
    function void print();  
        $display("a=%04",a);  
    endfunction  
    constraint a-c {  
        a dist {[10:50],[51:100],  
                [101:150]};  
    }  
endclass
```

```

Module tb;
  sample s;
  initial begin
    s = newc();
  repeat(10) begin
    s.randomize();
    s.print();
  end
end
endmodule

```

```

Dist = range.sv;
class sample;
  rand int a;
  function void print();
    $display("a=%0d", a);
  endfunction
  constraint a_c {
    a dist { [10:50]: 1/2,
              [51:100]: 1/5,
              [101:150]: 1/3 };
  }
endclass

```

```

Module tb;
  sample s;
  initial begin
    s = newc();
  repeat(10) begin
    s.randomize();
    s.print();
  end
end
endmodule

```

o/p: a=96
 a=133
 a=110
 a=65
 a=71
 a=87
 a=11
 a=2112

(3) if-else statement constraint
 Basing on the condition the constraint will work
 syntax: constraint constraint-name {
 if (condition)
 else if (condition)
 else
 }

```

if-else.sv
class sample;
  rand int a;
  function void print();
    $display("a=%0d, b=%0d", a, b);
  endfunction
  constraint a_c {
    b inside { [-5:5] };
    if (b > 0) a inside { [10:20] };
    else if (b < 0) a inside { [-20:-10] };
    else a = 0;
  }
endclass

```

```

Module tb;
  sample s;
  initial begin
    s = newc();
  repeat(10) begin
    s.randomize();
    s.print();
  end
end
endmodule

```

o/p: b=2, a=13
 b=3, a=10
 b=-2, a=-16
 b=4, a=12
 b=0, a=0
 b=-3, a=-18

if-else1.sv Q. Consider an eth-pkt class pkt-type is
small means len should be 10 to 20, medium - 50 to 100, large 101-200

if-else1.sv

```
typedef enum bit [1:0] { SMALL=2'b00, MEDIUM, LARGE } pkt-type;
```

```
class sample;
  rand int len;
  rand pkt-type pkt-t;
  function void print();
    $display("pkt-t=%s, len=%d",
      pkt-t, len);
  endfunction
  constraint a-c {
```

```
    if(pkt-t == SMALL) len
```

```
      inside { [10:20] };
```

```
    else if (pkt-t == MEDIUM)
```

```
      len inside { [50:100] };
```

```
    else len inside { [101:200] };
```

```
      pkt-t != 2'b11;
```

```
    }
```

```
  endclass
```

```
module tb;
```

```
  sample s;
```

```
  initial begin
```

```
    s = new();
```

```
    repeat (10) begin
```

```
      s.randomize();
```

```
      s.print();
```

```
    end
```

```
  end
```

```
endmodule
```

O/p: pkt-t = MEDIUM, len = 69

pkt-t = LARGE, len = 124

pkt-t = MEDIUM, len = 88

pkt-t = LARGE, len = 152

pkt-t = SMALL, len = 18

④ implicit constraint

- This constraint is same as if-else constraint.
- This constraint also works basing on the condition
- In implicit constraint we need to consider implicit operator (→)

Syntax:

```
constraint constraint-name {
```

```
  (condition1) → (logic1);
```

```
  (condition2) → (logic2);
```

```
  (condition3) → (logic3);
```

```
  (condition4) → (logic4);
```

```
}
```

Implicit.sv

```
class sample;
```

```
  rand int a, b;
```

```
  function void print();
```

```
    $display("b=%d, a=%d", b, a);
```

```
  endfunction
```

```
  constraint a-c {
```

```
    b inside { [-5:5] };
```

```
    (b > 0) → (a inside { [10:20] });
```

```
    (b < 0) → (a inside { [-20:-10] });
```

```
    (b == 0) → (a == 0);
```

```
  }
```

```
endclass
```

```
module tb;
```

```
  sample s;
```

```
  initial begin
```

```
    s = new();
```

```
    repeat (10) begin
```

```

s.randomize();
s.print();
end
end
endmodule

```

o/p: $b = -5, a = -10$
 $b = -3, a = -15$
 $b = -4, a = -13$
 $b = 1, a = 12$
 $b = 0, a = 0$
 $b = 2, a = 15$

⑤ Iterative constraint

- for looping statement we need to write the constraint.

- consider only foreach in iterative constraint

Syntax: constraint constr_name {
 foreach (array_name[i]) {
 // iterations
 }
}

Q: consider an array of size 10
 I want to assign random values b/w 20 to 50 to that array

Iterative: 50

```

class sample;
  rand int arr[10];
  function void print();
    $display("arr = %p", arr);
  endfunction
  constraint a-c {
    foreach (arr[i]) {
      arr[i] inside { [20:50] };
    }
  }
endclass

```

Module tb;

sample s;

initial begin

s=new();

s.randomize();

s.print();

end

endmodule

o/p: arr = { 41, 38, 30, 35, 39, 35, 47,
 43, 34, 44 }

Q: 0 5 0 5 0 5 0 5

class sample;

rand int arr[10];

function void print();

\$display("arr = %p", arr);

endfunction

constraint a-c {

foreach (arr[i]) {

if (i/2 == 0) arr[i] == 0;

else arr[i] == 5;

}

}

endclass

Module tb

Q: 0 1 0 2 0 3 0 4 0 5 0

Logic: if (i/2 == 0) arr[i] == 0;

else arr[i] = (i+1)/2;

Q: consider an array and generate unique random value b/w 10 to 30 using constraints

⑥ soft constraint:

- whenever constraint conflict will be happened that time we need to make one constraint as soft, so it will consider only the main constraint
- Basing on the main constraint it will generate the random values.
- we use soft keyword for mentioning the soft constraint.
- if we mention both the constraint as soft means it will give priority to inline constraint.

Ex: constraint constraint_name {

 // soft constraint logic

 }

endclass

Module tb;

 soft-type sv

class sample;

 rand int a;

 function void print();

 \$display("a=%0d", a);

 endfunction

 constraint a-c {

 a inside {[20:30]};

 }

endclass

module tb;

 sample s;

 initial begin

 s = new();

 re s.randomizec() with { soft a = 50; };

 s.print();

 end

endmodule

Soft-type3.sv

class sample;

 rand int a;

 function void print();

 \$display("a=%0d", a);

 endfunction

 constraint a-c {

 soft a inside {[20:30]};

 }

endclass

soft-type2.sv

class sample;

 rand int a;

 function void print();

 \$display("a=%0d", a);

 endfunction

 constraint a-c {

 soft a inside {[20:30]};

 }

endclass

module tb;

 sample s;

 initial begin

 s = new();

 s.randomizec() with { a = 50; };

 s.print();

 end

endmodule

module tb;

 sample s;

 initial begin

 s = new();

 repeat (10) begin

 s.randomizec() with { soft a = 50; };

 s.print();

 end

 end

endmodule

Unique constraint:

It is used to generate the unique values

Syntax: constraint constraint-name {

unique {value1, value2, value3, ...} }

Unique.SV:

```
class sample;
rand void print;
rand int arr[10];
function void print();
$display("arr=%p", arr);
endfunction
constraint a-c {
foreach (arr[i]) {
arr[i] inside {[10:30]};
foreach (arr[i]) {
if (i != j) arr[i] != arr[j];
}
}
}
endclass
module tb;
sample s;
initial begin
s = new();
s.randomize();
s.print();
end
endmodule
```

unique2.SV

```
class sample;
rand int arr[];
function void print();
$display("arr=%p", arr);
endfunction
function new();
arr = new[10];
endfunction
constraint a-c {
foreach (arr[i]) {
```

Unique1.SV:

```
class sample;
rand int a,b,c;
function void print();
$display("a=%d, b=%d, c=%d", a, b, c);
endfunction
constraint a-c {
a inside {[10:50]};
b inside {[10:50]};
c inside {[10:50]};
Unique {a,b,c};
}
endclass
module tb;
sample s;
initial begin
s = new();
repeat (10) begin
s.randomize();
s.print();
end
end
endmodule
```

```
arr[i] inside {[10:50]};
```

```
unique {arr};
```

```
}
```

```
}
```

```
endclass
```

```
module tb;
```

```
sample s;
```

```
initial begin
```

```
s = new();
```

```
repeat (10) begin
```

```
s.randomize();
```

```
s.print(); end
```

⑧ Variable order constraint:

- In variable order we need to specify the order in variable we have two keywords "solve" and "before"

Variable-order.sv

```
class sample;
  rand int a,b;
  function void print();
    $display("a=%d,b=%d",a,b);
  endfunction
```

```
  constraint a-c {
    b inside {-5:5};
    (b>0) -> (a>0);
    (b<0) -> (a<0);
    (b==0) -> (a==0);
    solve (a) before (b);
  }
endclass
```

```
module tb;
  sample s;
  initial begin
    s=new();
    s.randomize();
    s.print();
  end
endmodule
```

variable-order2.sv

```
class sample;
  rand int a,b,c;
  function void print();
    $display("a=%d, b=%d, c=%d",
            a,b,c);
  endfunction
```

```
  constraint a-c {
    a inside {-5:5};
    b inside {-5:5};
    c inside {-5:5};
    (b>0) -> (a>0);
    (b<0) -> (a<0);
    (b==0) -> (a==0);
    (c>0) -> (b>0);
    (c<0) -> (b<0);
    (c==0) -> (b==0);
    solve c before b;
    solve b before a;
  }
endclass
```

→ module pool.

Q. consider a 16 bit vector, in that vector there is no consecutive 1s.

Vect.sv

```
class sample;
  rand bit [15:0] a;
  function void print();
    $display("a=%b",a);
  endfunction
  constraint a-c {
    foreach (a[i]) {
      if (i>0) {
        if (a[i]==1) {
          a[i-1] != 1; } }
    }
  }
endclass
```

module tb;

```
  sample s;
  initial begin
    s=new();
    repeat (5) begin
      s.randomize();
      s.print();
    end
  end
endmodule
```

o/p: a = 00100101000100

b = 010010101001

c = 1001000100010001

Queues in constraints:

queue_name.size() == <value>;

Queue.sv

```
class sample;
    rand int que[5];
    function void print();
        $display("que = %p", que);
    endfunction
    constraint a-c {
        que.size() == 10;
        foreach (que[i]) {
            que[i] inside {50:100};
        }
    }
endclass
```

```
module tb;
    sample s;
    initial begin
        s = new();
        s.randomize();
        s.print();
    end
endmodule
```

Queue1.sv

```
class sample;
    rand int que[5];
    function void print();
        $display("que = %p", que);
    endfunction
    constraint a-c {
        que.size() inside {1:10};
        foreach (que[i]) {
            que[i] inside {50:100};
        }
        foreach (que[i]) {
            if (i != j) que[i] != que[j];
        }
    }
endclass
```

```
module tb;
    sample s;
    initial begin
        s = new(); repeat (10) begin
            s.randomize();
            s.print(); end
    end
endmodule
```

constraints in inheritance: constraints will work in child class

constraint-inher.sv

```
class parent;
    rand int a;
    function void print();
        $display("a = %d", a);
    endfunction
    constraint a-c {
        a inside {10:50};
    }
endclass
```

class child extends parent;

```
    rand int b;
    function void print();
        super.print();
        $display("b = %d", b);
    endfunction
    constraint b-c {
        b inside {50:100};
    }
endclass
```

```

Module tb;
  child ch;
  initial begin
    ch=newC();

```

```

    ch.randomize();
    ch.print();

```

```

end
endmodule

```

```

constraint-override

```

```

class parent;

```

```

  rand int a;

```

```

  function void print();

```

```

    $display("a=%d", a);

```

```

  endfunction

```

```

constraint a-c{

```

```

  a inside { [10:50] };

```

```

}

```

```

endclass

```

```

class child extends parent;

```

```

  rand int b;

```

```

  function void print();

```

```

    super.print();

```

```

    $display("b=%d", b);

```

```

  endfunction

```

```

  constraint b-c{

```

```

    b inside { [50:100] };

```

```

}

```

```

  constraint a-c{

```

```

    a inside { [100:200] };

```

```

}
endclass

```

```

Module tb;

```

```

  child ch;

```

```

  initial begin

```

```

    ch=newC();

```

```

    ch.randomize();

```

```

    ch.print();

```

```

  end

```

```

endmodule

```

Note: override the constraints in child class but we need to consider the same name constraint name.

* Constraints are virtual in nature so these will be applicable in polymorphism

```

constraint - poly.sv

```

```

class parent;

```

```

  rand int a;

```

```

  virtual function void print();

```

```

    $display("a=%d", a);

```

```

  endfunction

```

```

  constraint a-c{

```

```

    a inside { [100:200] };

```

```

}

```

```

endclass

```

```

class child extends parent;

```

```

  rand int b;

```

```

  function void print();

```

```

    super.print();

```

```

    $display("b=%d", b);

```

```

  endfunction

```

```

  constraint b-c{

```

```

    b inside { [10:50] };

```

```

}
endclass

```

```

Module tb;

```

```

  child ch;

```

```

  parent p;

```

```

  initial begin

```

```

    ch=newC();

```

```

    ch.randomize();

```

```

    $cast(p, ch);

```

```

    p.print();

```

```

  end

```

```

endmodule

```

Q. Write a constraint for 0, 9, 99, 999, 9999, 99999

① $0, 9, 99, 999, 9999 \Rightarrow (10^i - 1) \rightarrow i=0 \rightarrow 10^0 - 1 = 0$
 $(10^1 - 1) (10^2 - 1) (10^3 - 1) (10^4 - 1)$
 $i=1 \rightarrow 10^1 - 1 = 9$
 $i=2 \rightarrow 10^2 - 1 = 99$

// pattern 0, 9, 99, 999, 9999

class sample

rand int a[10];

function void print();

\$display("a=%p", a);

endfunction

constraint a- {

foreach(a[i]) {

a[i] == (10 * i - 1);

}

} endclass

module tb;

sample s;

initial begin

s = new;

s.randomize();

s.print();

end

endmodule

Slave sv

class common

static bit [4:0] slave_q[3];

static bit [4:0] value;

endclass

class sample;

rand int addr;

rand int ssel;

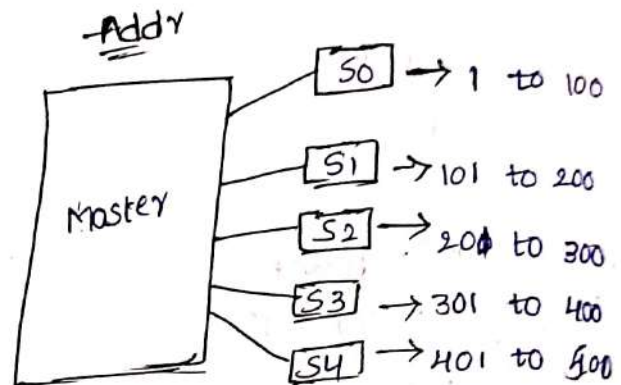
function void print();

\$display("ssel=%0d, addr=%0d", ssel, addr);

endfunction

constraint a- {

ssel inside {common::slave_q};



{0, 1, 1, 3}

S1 → 101 to 200

S3 → 301 to 400

0, 1, 2, 3, 4, 5 → 5 bit

0 0 1 0 1 → 0, 2
 2 0
 0 0 0 0 1
 0 0 0 1 0
 0 0 1 0 0
 0 1 0 0 0
 1 0 0 0 0

0 1 1 1 0 → ssel → {0, 2}

value → 1 0 1 1 0 → {1, 2, 3}

if(ssel == 0) addr inside {1:100};

if(ssel == 1) addr inside {101:200};

if(ssel == 2) addr inside {201:300};

if(ssel == 3) addr inside {301:400};

if(ssel == 4) addr inside {401:500};

endclass

module tb;

sample s;

initial begin

```

// $value $plusargs ("value = %0b", common::value);
common::value = 5'b10110;
for (int i=0; i<1; i++) begin
  if (common::value[i] == 1) begin
    common::slaveq.push-back(i);
  end
end
$display ("slaveq = %p", common::slaveq);
S = new(1);
repeat (10) begin
  S.randomize();
  S.print();
end
end
endmodule

```

S - 31

* why we are considering the test-bench?

generating the inputs, applying the inputs to the design and checking the actual values with expected values

* If we get 100% function coverage, 100% code coverage and 100% assertions

* coverage means it tells about how much percentage will be covered in the complete series.

• coverage must be in percentages

types of coverage: 1. functional coverage 2. code coverage

① Functional coverage: This functional coverage will check the applicable for the test bench signals.

Functional coverage will check the testbench signals covered and not.

② code coverage: This code will be applicable for design file.

code coverage will check coverage for "FCBEST"

* F - FSM coverage

C - Conditional coverage

B - Branch coverage

E - Expression

S - Statement

T - Toggle

Functional coverage: functional coverage will follow some hierarchy

coveragegroup coveragegroup_name;

coverpoint1 coverpoint_name {

bin1

bin2

bin3, ... }

coverpoint2 coverpoint_name { bin1

bin2

bin3, ... }

coverpoint3 coverpoint_name { bin1

bin2

bin3, ... }

endgroup

Covergroup: used to group all the coverpoints

syntax: coveragegroup coveragegroup_name @(event-control);

endgroup

coverpoints: In coverpoint select the signals

syntax: coverpoint coverpoint_name {

// bins

}

Bins: used to create the possibilities

16 → 4 bins

syntax: bins bin_name = { <value> };

1 bins 0 to 3 - 100%

Memory coverpoints:

2 " 4-7

3 " 8-11

4 " 12-15 } 100%

1. wr-rd 2. addr

coveragegroup for memory:

coveragegroup cb;

coverpoint tx.wr-rd {

bins WRITES = {1'b1};

bins READS = {1'b0};

} endgroup

10 → 4098

addr-write

addr-reads

wr-rd

2 bins

addr

4 bins → write

=> 8 bins

chip

100% → 99%

* How create bins

Types of functional coverages

instance coverage
coverage options

addr

rand int value

constraint a-c {

value inside $\{0, 10\}$;

addr = $2^{**}a$;

\$countones(addr) == 1;

\$onehot(addr) =

16 bit $\Rightarrow$ 001000
8

\$countones(que[i]) == 1

Types of bins: 1. Bins

2. Illegal

3. Ignore

1. Bins: Normal bins which are

involved in the percentage
calculations (% calculations);

Syntax: bins bin-name = {<value or
range>;

2. Illegal bins: These are unwanted

bins which are not involved
in the percentage calculations

Syntax: Illegal-bins bin-name =

{<value or range>;

3. Ignore bins: the bins which are

ignored, these bins are not
involved in the percentage
calculations

Syntax: Ignore-bins bin-name =

{<value or range>;

Blood test

Sugar test

bp

sugar test

Low $\rightarrow$ 50 - 90

Normal $\rightarrow$ 90 - 120

High $\rightarrow$ 120 - 250

above 250 - 500 $\rightarrow$ Illegal

below 50 or (0 - 50)

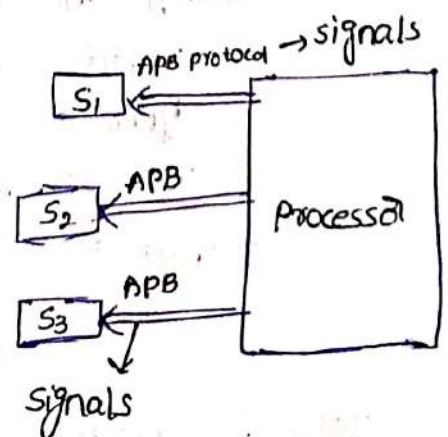
$\rightarrow$ Ignore

bin0 $\rightarrow$ bin63

1 - bin 0 to 63

0 to 62 $\rightarrow$ 1

63 $\rightarrow$



write the covergroup for apb_tx class

-30

coverpoint ssel {

bins slave0 = {4'b0000};

slave1 = {4'b0001};

slave2 = {4'b0010};

slave3 = {4'b0100};

slave4 = {4'b1000};

ignore-bins none = {4'b0011, 4'b101, 4'b110, 4'b111, 4'b1001, 4'b1010, 4'b1011, 4'b1100, 4'b1101, 4'b1110, 4'b1111}; }

How many ways we can create bins: 3 ways.

1. Implicit

2. Explicit

3. By using option

1. Implicit bins creation: In implicit "No need" to create bins, basing on the signal size it will create the bins

* It will create only maximum 64 bins

* Having more size, in the last bin it will assign all the left over values.

syntax: coverpoint coverpoint_signal_name { }

2. Explicit bins creation: Need to create the bins manually
coverpoint coverpoint_name {

bins WRITES = 1'b1;

bins READS = 1'b0;

}

for creating the bins explicitly we need to consider variables

static arrays

dynamic arrays

we can pass the condition

coverpoint a { bins A = {10, [20:30], 13, 14}; // values will be 13 and bits will be 14 }

//static arrays:

coverpoint a {

bins A[5] = {10, [20:30], 13, 14, 15}; // values will be 14
And bins will be 5 }

if we consider dynamic array it will create maximum 64 bins by using option

we can create the bins by using the option

✓ syntax: option. auto-bin-max = <value>;

By using option

22/03/25 ~~33~~ 33

Instance coverage: we will create multiple instance for coveragegroup

Transition coverage: Transition coverage is applicable for fsm

In coverage transition can be represented as " $\Rightarrow$ "

typedef enum bit [5:0] {s0=0, s1, s2, s3, s4, s5} state-t;

• will check about the variable changes from one value to other

Module overlapping: 10110 $\rightarrow$ 6 states

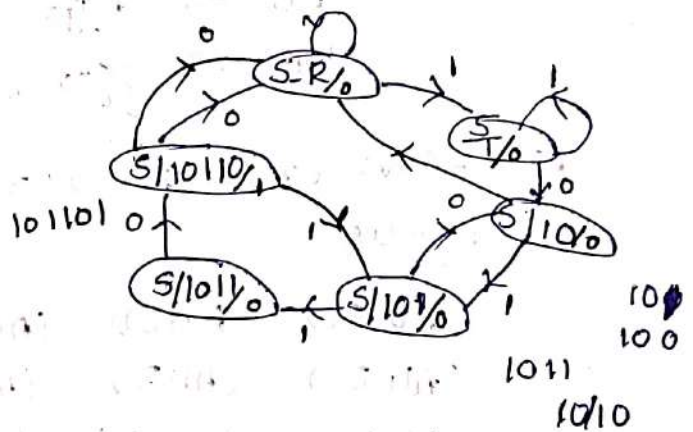
s0 $\Rightarrow$ s1, s1 $\Rightarrow$ s2, s2 $\Rightarrow$ s3

s3 $\Rightarrow$ s4, s4 $\Rightarrow$ s5, s5 $\Rightarrow$ s3

s5 $\Rightarrow$ s0, s4 $\Rightarrow$ s1, s3 $\Rightarrow$ s2

s2 $\Rightarrow$ s0, s0 $\Rightarrow$ s1 $\Rightarrow$ s2

(s0 $\Rightarrow$ s1 $\Rightarrow$ s2 $\Rightarrow$ s3 $\Rightarrow$ s4)



class cov;

rand state-t s-t;

coveragegroup cov;

coverpoint s-t;

bins b[] = (s0 $\Rightarrow$ s1), (s1 $\Rightarrow$ s2), (s2 $\Rightarrow$ s3), (s3 $\Rightarrow$ s4), (s4 $\Rightarrow$ s5),
(s5 $\Rightarrow$ s0), (s5 $\Rightarrow$ s3), (s4 $\Rightarrow$ s1), (s3 $\Rightarrow$ s2), (s2 $\Rightarrow$ s0);

illegal bins a[] = (s3 $\Rightarrow$ s3), (s2 $\Rightarrow$ s2);

bins other-sequence = default sequence;

}

endgroup

function newc();

cov = newc();

endfunction

task runc();

cov.sample();

endtask

endclass

Module tb;

cov c;

initial begin

c = newc();

repeat(50) begin

c.randomize();

c.run();

end

end

endmodule

Cross coverage: If we have 2 or more coverpoints we can perform the cross coverage for that coverpoints. "Cross" keyword
covergroup cb;

A-COV: coverpoint a {

bins A0 = {[10:20]};

bins A1 = {[30:40]};

bins A2 = {[50:60]};

bins A3 = {[70:80]};

B-COV: coverpoint b {

bins B0 = {[10:20]};

bins B1 = {[30:40]};

bins B2 = {[50:60]};

bins B3 = {[70:80]};

CROSS-COV: CROSS A-COV, B-COV;

endgroup

(A0, B0) (A0, B1) (A0, B2) (A0, B3)

(A1, B0) (A1, B1) (A1, B2) (A1, B3)

(A2, B0) (A2, B1) (A2, B2) (A2, B3)

(A3, B0) (A3, B1) (A3, B2) (A3, B3)

Ex:

x: coverpoint x {

bins x0 = {[1:5]};

x1 = {[6:10]};

x2 = {[11:15]};

x3 = {[16:20]};

y: coverpoint y {

bins y0 = {[1:5]};

[6:10];

[11:15];

bins y3 = {[16:20]};

The cross coverage will generate 16 bins

(x0, y0) (x0, y1) (x0, y2) (x0, y3)

(x1, y0) (x1, y1) (x1, y2) (x1, y3)

(x2, y0) (x2, y1) (x2, y2) (x2, y3)

(x3, y0) (x3, y1) (x3, y2) (x3, y3)

Options in coverage:

1. weight

2. Name

3. comment

4. per-instance

5. Goal

6. AtLeast

7. detect overlap

8. auto-bin-max

1. weight: Basing on the weightage it will calculate the percentages

Ex: 3 coverpoints, wr-td, cross

wr-td = 100% coverage

add r = 40% coverage

cross = 30% coverage

average percentage = $\frac{(100+40+30)}{3}$

average percentage = 57%

for wr-td weightage will be 5;

" dddr " " " 2

" cross " " " 1

$$\text{average coverage \%} = \frac{(\text{weightage of CP1} * \text{coverage \% of CP1}) + (\text{" " CP2} * \text{" " CP2}) + \dots}{\text{weightage of CP1} + \text{weightage of CP2} + \dots}$$

$$\text{average coverage \%} = \frac{(5 * 100) + (2 * 40) + (1 * 30)}{5 + 2 + 1} = 76.25\%$$

syntax: option-weight = <value>;

* weight is applicable for covergroup, coverpoint, cross

② Name: used to give a name to the covergroup

Name is only applicable for covergroup

syntax: option-name = "string";

③ comment: using for writing the comments

comment is applicable for covergroup, coverpoint and cross

syntax: option.comment = "comments"

④ per-instance: used to create the instance to the covergroup

syntax: option.per-instance = 1;

⑤ Goal: tells about the colour of the coverpoint

<50 → RED colour

>50 and <90 → YELLOW colour

>90 → GREEN colour

syntax: option.goal = 50;

⑥ At-least: whenever we mentioned a value in atleast that many time the bin need to be hits

wr-td {
 option.atleast = 10;

syntax: option.atleast = 5;

7. detect overlap: it tells about that if there is any overlap is happened b/w the coverpoint

syntax: option. detect-overlap = 1;

8. auto-bin-max: used to creating the bins

syntax: option. auto-bin-max = <value>

cross coverage filters: 2 filters

1. Bins of
2. Bins of intersect

1. Bins of: it will filter the bins basing the bin name

x: coverpoint x {

2. Bins of intersect: used to filter the bins basing the values

Bins of intersect:

x: coverpoint x {

bins x0 = {1:5};

bins x1 = {6:10};

bins x2 = {11:15};

bins x3 = {16:20};

y: coverpoint y {

bins y0 = {1:5};

bins y1 = {6:10};

bins y2 = {11:15};

bins y3 = {16:20};

Cross-Bins: cross x, y {

bins a = bins of (x) intersect ([5:10]);

bins b = bins of (y) intersect ([10:17]);

}

o/p: (x0, y0) (x0, y1) (x0, y2) (x0, y3)

(x1, y0) (x1, y1) (x1, y2) (x1, y3)

(x0, y1) (x1, y1) (x2, y1) (x3, y1)

(x0, y2) (x1, y2) (x2, y2) (x3, y2)

(x0, y3) (x1, y3) (x2, y3) (x3, y3);

Bins of

x: coverpoint x {

bins x0 = {1:5};

bins x1 = {6:10};

bins x2 = {11:15};

bins x3 = {16:20};

y: coverpoint y {

bins y0 = {1:5};

bins y1 = {6:10};

bins y2 = {11:15};

bins y3 = {16:20};

Cross-Bins: cross x, y {

bins b = bins of (x1);

bins a = bins of (y2);

bins c = bins of (x3) &

bins of (y3);

bins d = bins of (x0) ||

bins of (y0);

o/p: (x1, y0) (x1, y1) (x1, y2) (x1, y3)

(x0, y0) (x1, y2) (x2, y2) (x3, y3)

(x3, y3)

(x0, y0) (x0, y1) (x0, y2) (x0, y3)

(x1, y0) (x2, y0) (x3, y0);